Join at slido.com
#2474272

Course Evaluation

2474272

The final course survey is intended to help us improve future offerings of the course.

If you complete our **internal anonymous survey AND the official Course Evaluation** (http://course-evaluations.berkeley.edu) **you will get 0.5% extra credit points** to your grade. If **90%** of the class also submits both, then everyone who **submitted both** will receive **an additional 0.5% percentage point** on their final grade.

Internal course survey link: **https://bit.ly/cs189-eval**

LLM for Specific Tasks

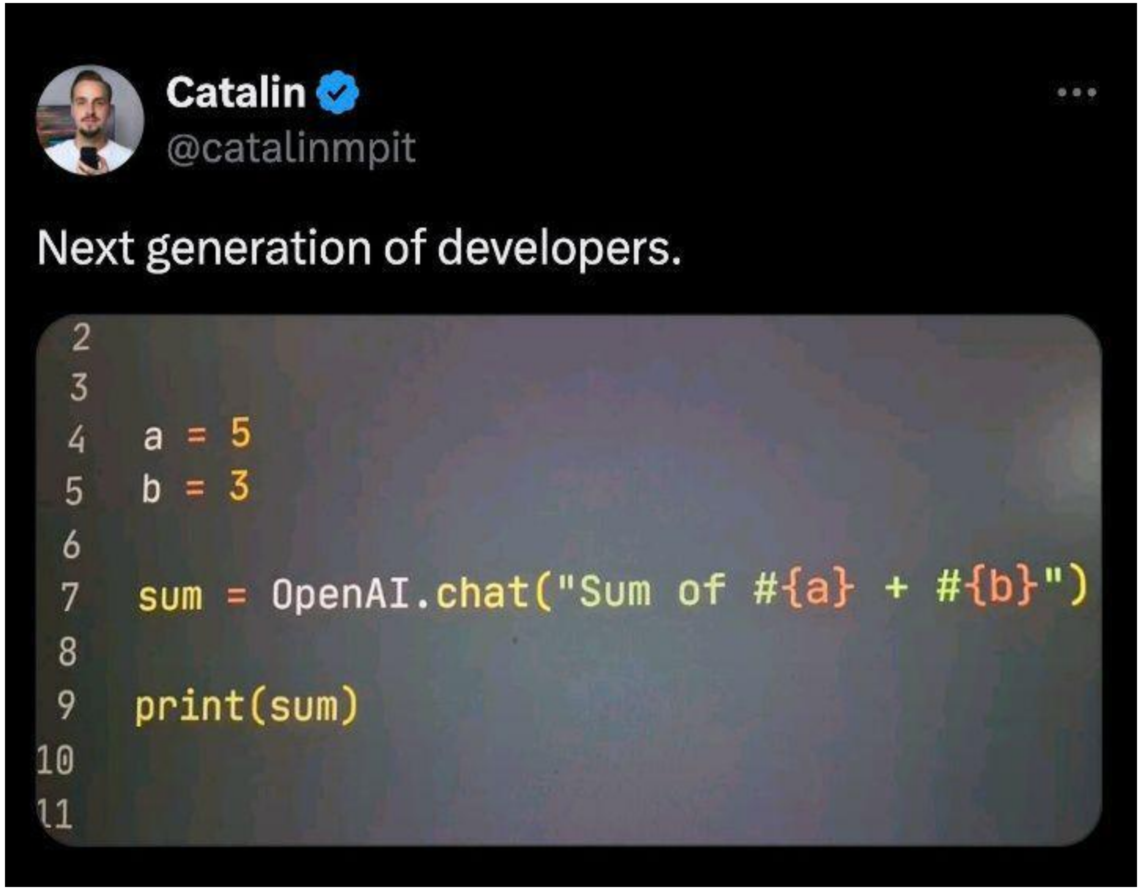
2474272

July 21, 2025 Research

Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad

Thang Luong and Edward Lockhart

Share



LLM for Specific Tasks

2474272

- **Supervised Fine Tuning (SFT):**
  Finetune a pre-trained model on a smaller, labeled dataset for a specific task

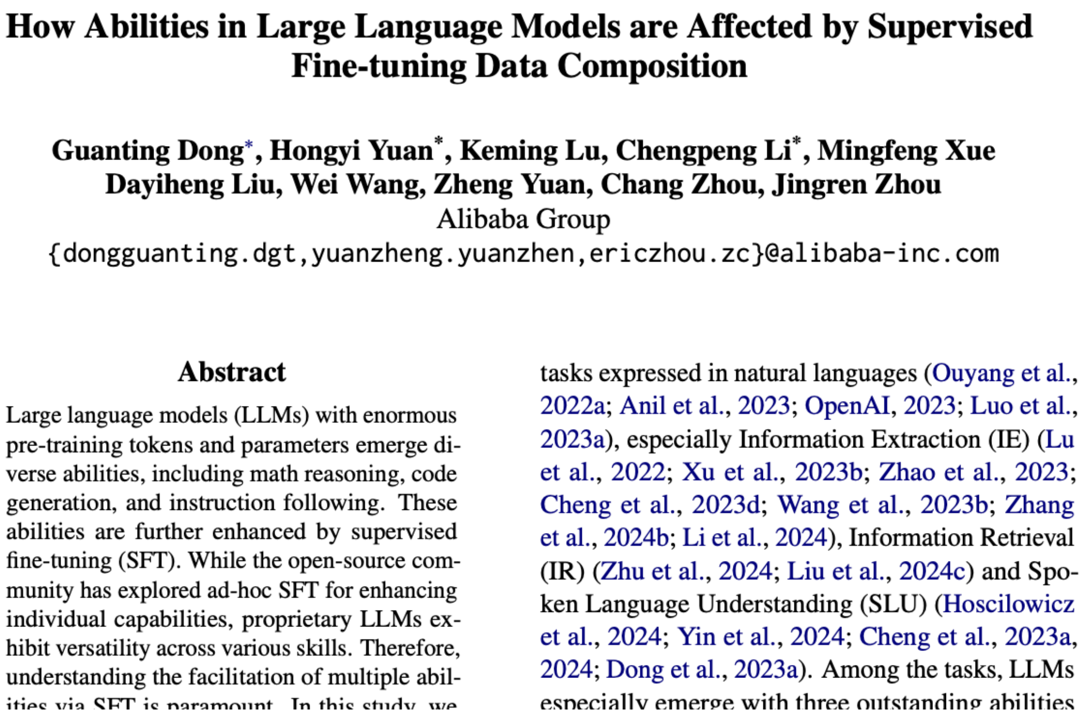
- **How do we achieve this without losing the ability to generalize?**

LLM for Specific Tasks

2474272

- **Supervised Fine Tuning (SFT):**
  Finetune a pre-trained model on a smaller, labeled dataset for a specific task

- **How do we achieve this without losing the ability to generalize?**



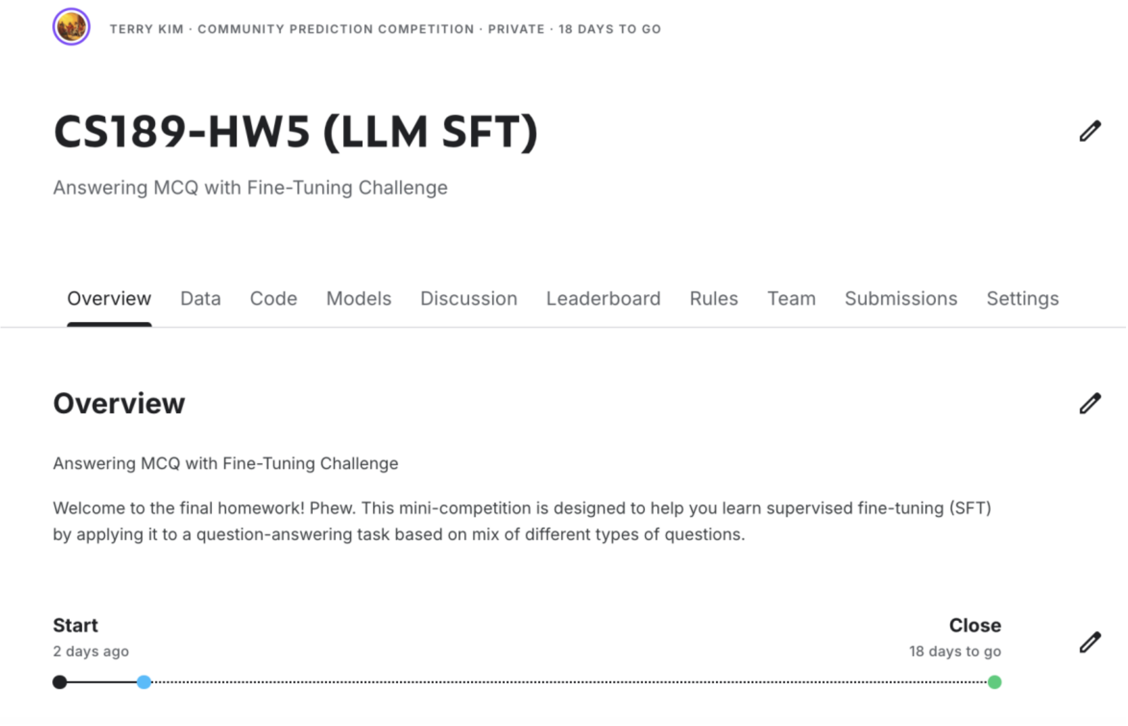
How Abilities in Large Language Models are Affected by Supervised Fine-tuning Data Composition

Guanting Dong*, Hongyi Yuan*, Keming Lu, Chengpeng Li*, Mingfeng Xue
Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, Jingren Zhou
Alibaba Group
{dongguanting.dgt,yuanzheng.yuanzhen,ericzhou.zc}@alibaba-inc.com

Abstract

Large language models (LLMs) with enormous pre-training tokens and parameters emerge diverse abilities, including math reasoning, code generation, and instruction following. These abilities are further enhanced by supervised fine-tuning (SFT). While the open-source community has explored ad-hoc SFT for enhancing individual capabilities, proprietary LLMs exhibit versatility across various skills. Therefore, understanding the facilitation of multiple abilities via SFT is paramount. In this study, we

tasks expressed in natural languages (Ouyang et al., 2022a; Anil et al., 2023; OpenAI, 2023; Luo et al., 2023a), especially Information Extraction (IE) (Lu et al., 2022; Xu et al., 2023b; Zhao et al., 2023; Cheng et al., 2023d; Wang et al., 2023b; Zhang et al., 2024b; Li et al., 2024), Information Retrieval (IR) (Zhu et al., 2024; Liu et al., 2024c) and Spoken Language Understanding (SLU) (Hoscilowicz et al., 2024; Yin et al., 2024; Cheng et al., 2023a, 2024; Dong et al., 2023a). Among the tasks, LLMs especially emerge with three outstanding abilities



Homework 5!

2474272

Due Date: December 12

- Paper Question
- Kaggle Submission (With Write-up)

Important!

- 3 Submissions; Paper Questions, Code [Notebook + PDF], Writeup
- Make sure you submit with proper format
- You can submit up to 5 Kaggle submissions per day

Lecture 25

Contrastive Learning and Diffusion Models

Contrastive representation learning and diffusion models

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

Slides adopted from Steve Seitz, Andrew Owens, etc.

Roadmap

- Discriminative
  - Rotation
  - Jigsaw
  - Clustering
  - Contrastive Learning
- Diffusion Models

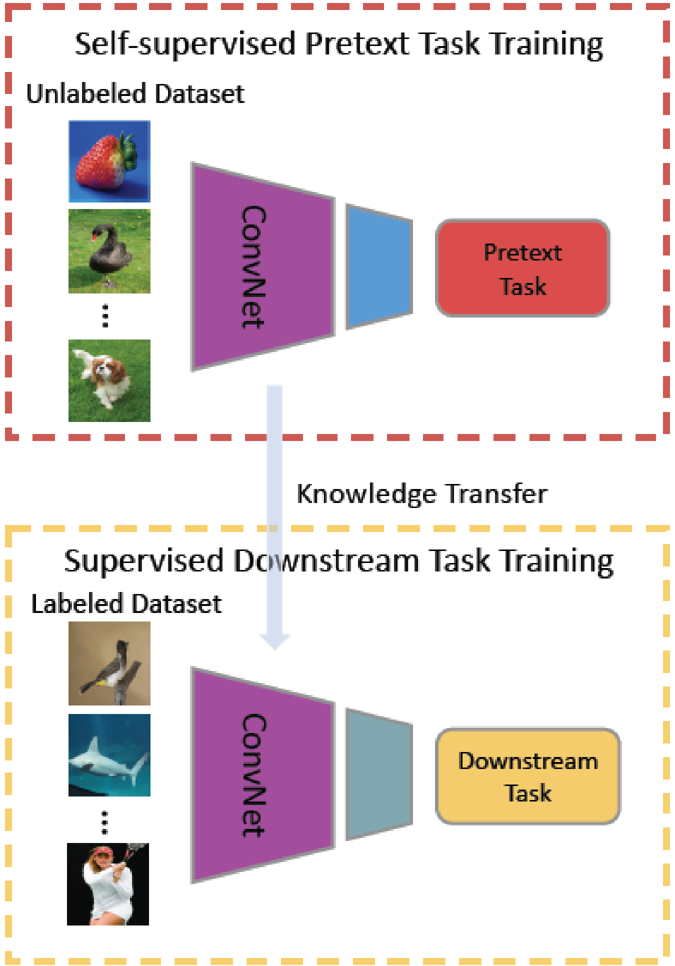
2474272

Self-Supervised Learning

2474272

- ***Unsupervised Learning:*** Model isn't told what to predict.

- ***Self-Supervised Learning:*** Representation learning with unlabeled data
  - Learn useful feature representations from unlabeled data through pre-text tasks.
  - The term "self-supervised" refers to creating its own supervision (i.e., without supervision, without labels)

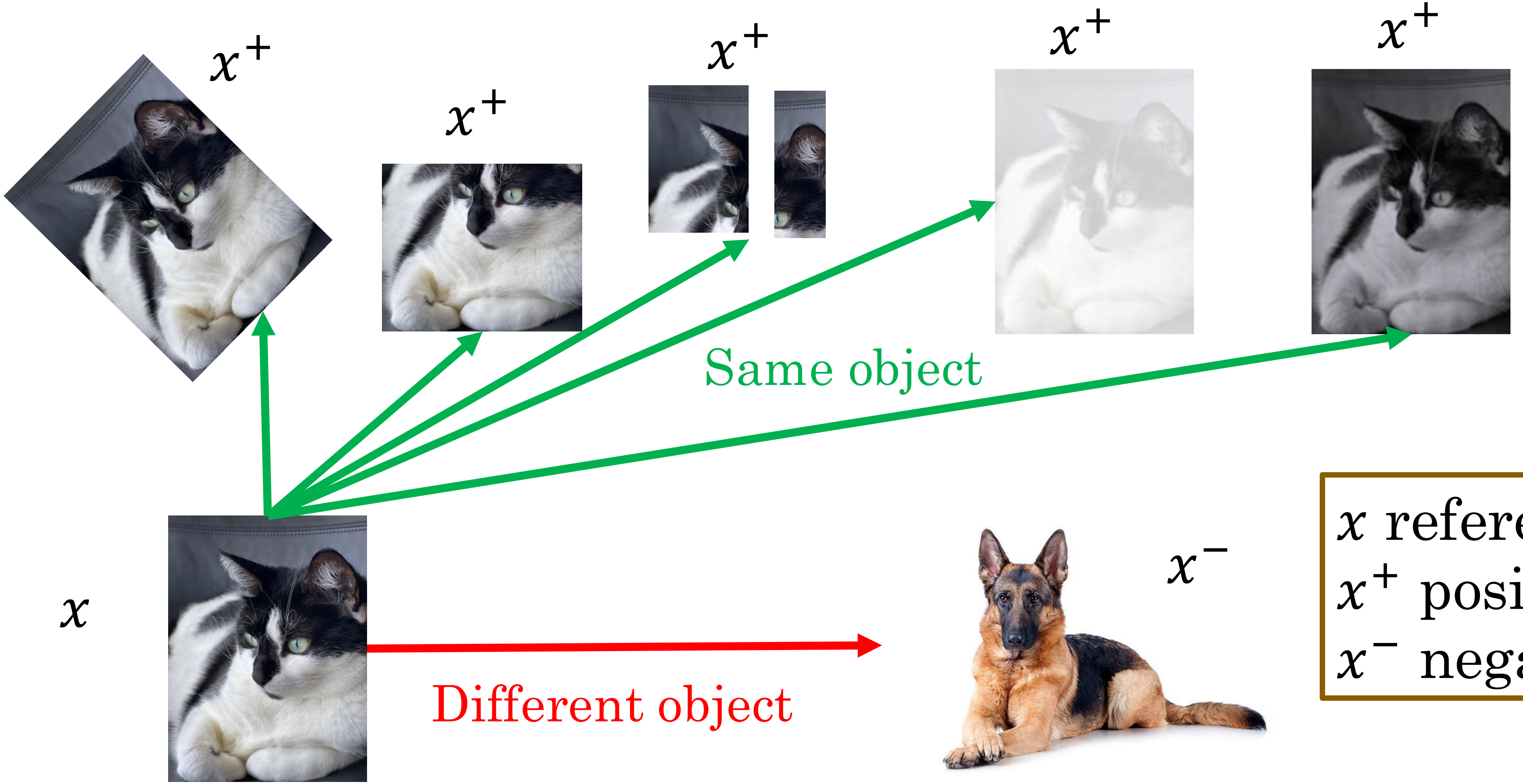


Contrastive Representation Learning

- Discriminative
  - Rotation
  - Jigsaw
  - Clustering
  - Contrastive Learning
- Diffusion Models

2474272

Contrastive Representation Learning



x reference
x^+ positive
x^- negative

Formulation for Contrastive Learning

2474272

- What we want:

$$\color{blue}{score}\left(f(x), f(x^+)\right) \gg \color{blue}{score}\left(f(x), f(x^-)\right)$$

x reference
x^+ positive
x^- negative

- Given a chosen score function, we aim to **learn an encoder function $\boldsymbol{f}$** that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-).

InfoNCE Loss

2474272

- InfoNCE (Information Noise-Contrastive Estimation) loss minimizes **the similarity** between an anchor sample and a set of negative (unrelated) samples.
- Given 1 positive sample and N-1 negative samples:

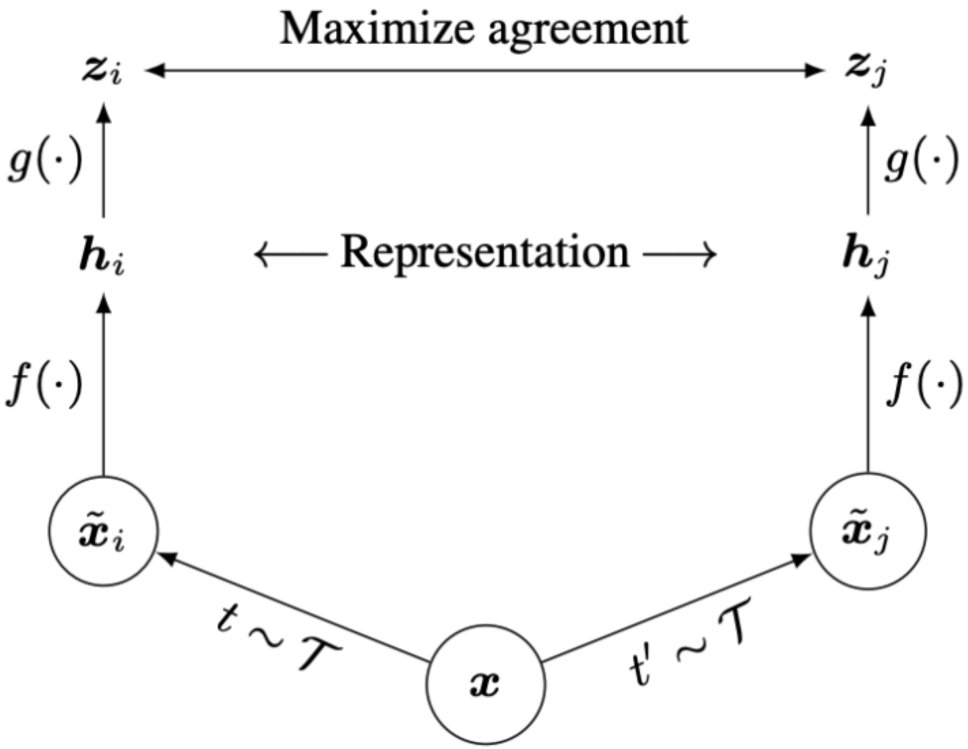
$$L = -\mathbb{E}_x \left[\log \frac{e^{s\left(f(x), f(x^+)\right)}}{e^{s\left(f(x), f(x^+)\right)} + \sum_{j=1}^{N-1} e^{s\left(f(x), f(x^-)\right)}} \right]$$

This is similar to cross-entropy loss for a N-way softmax classifier.

SimCLR: A Simple Framework for Contrastive Learning

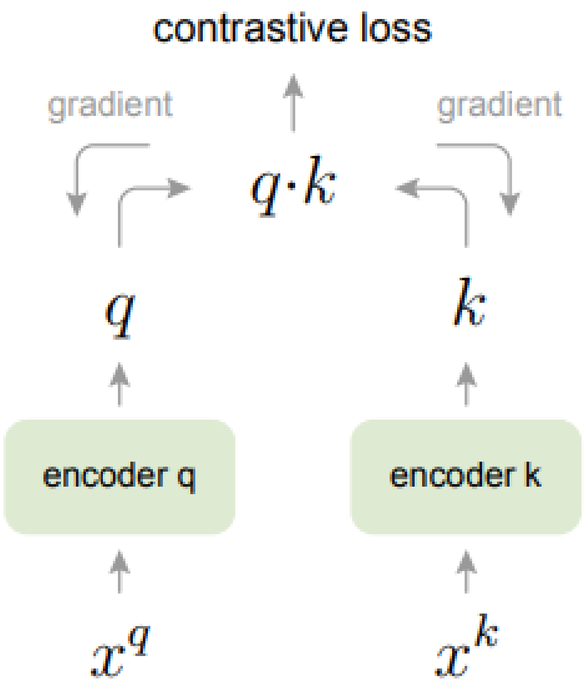
2474272

- Cosine similarity as the score function
- Use a projection network g(·) to project features to a space where contrastive learning is applied.
- Generate positive samples through data augmentation:
  - Random cropping, random color distortion, and random blur.



Momentum Contrastive Learning (MoCo)

2474272



(a) end-to-end

Decouples negative sample size from minibatch size; allows large batch training without TPUs.

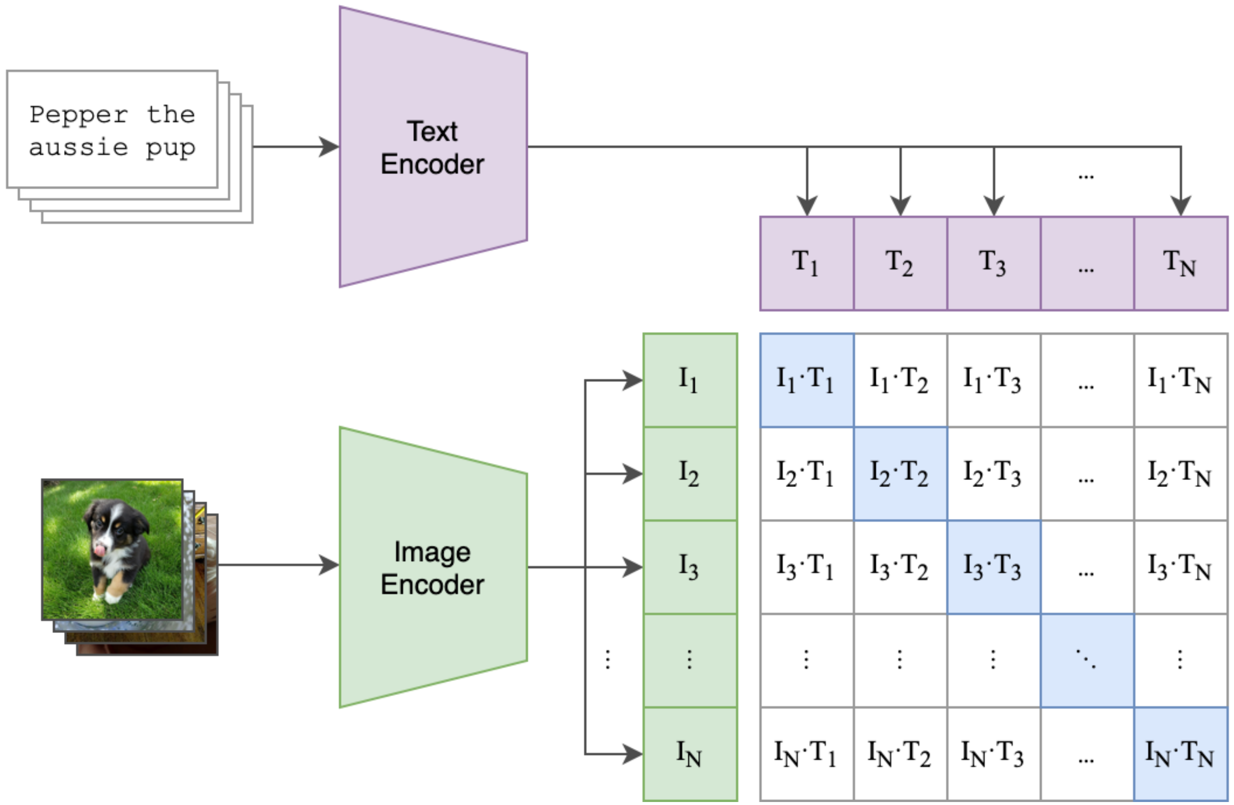
https://arxiv.org/abs/1911.05722

Contrastive Language Image Pre-Training (CLIP)

OpenAI

2474272

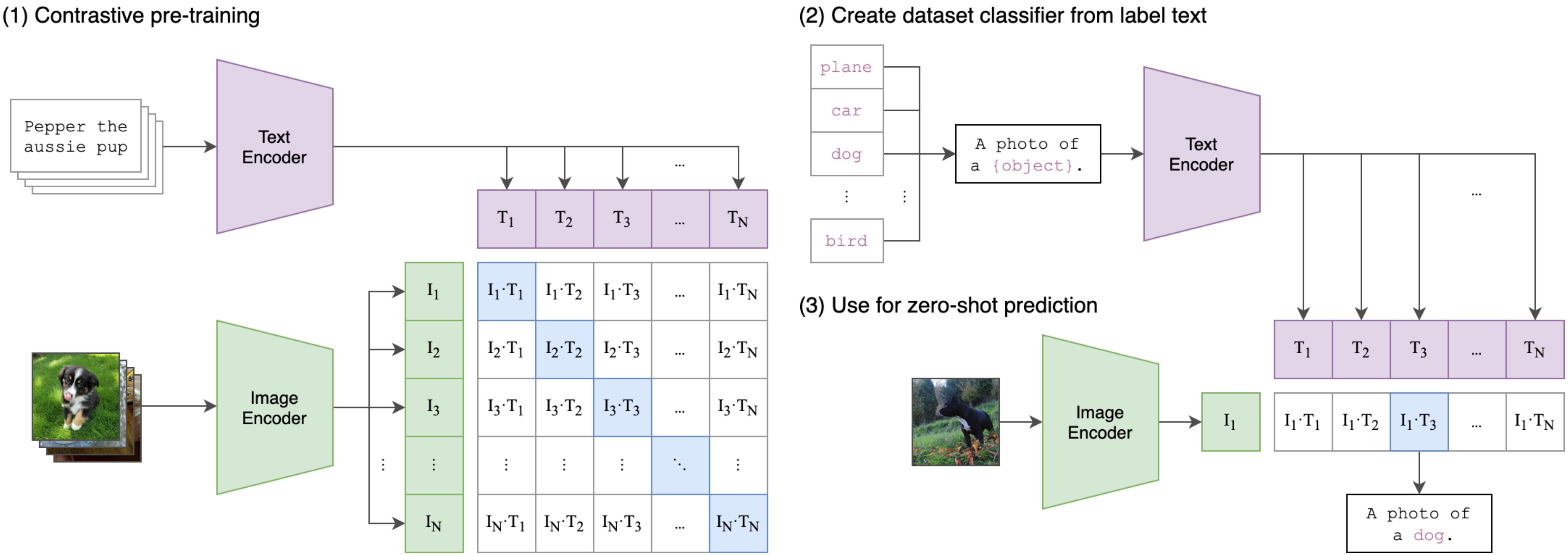
(1) Contrastive pre-training



$$\text{InfoNCE loss} = -\log \frac{\exp I_i T_i}{\sum_j \exp I_i T_j}$$

Contrastive Language Image Pre-Training (CLIP)

2474272



Do not edit
How to change the design

What is a key disadvantage of CLIP (and many other contrastive methods)?

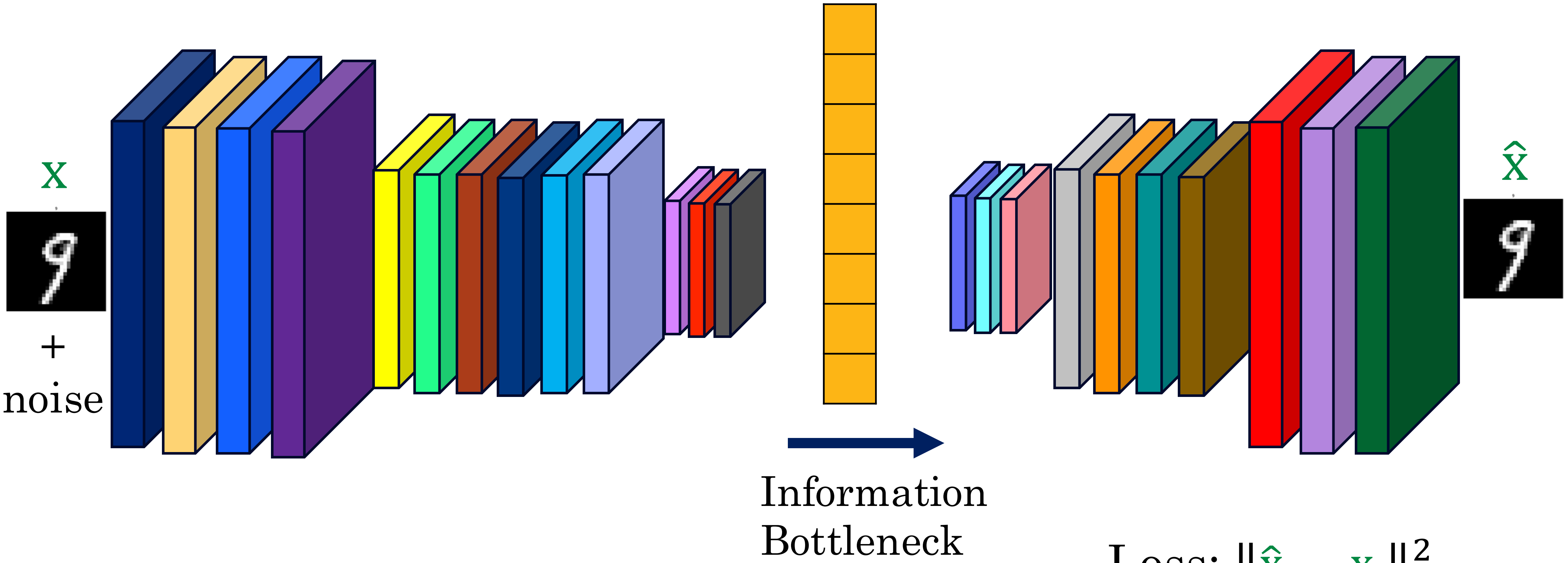
Diffusion Models

- Discriminative
  - Rotation
  - Jigsaw
  - Clustering
  - Contrastive Learning
- Diffusion Models

2474272

Recall the Denoising Problem

2474272



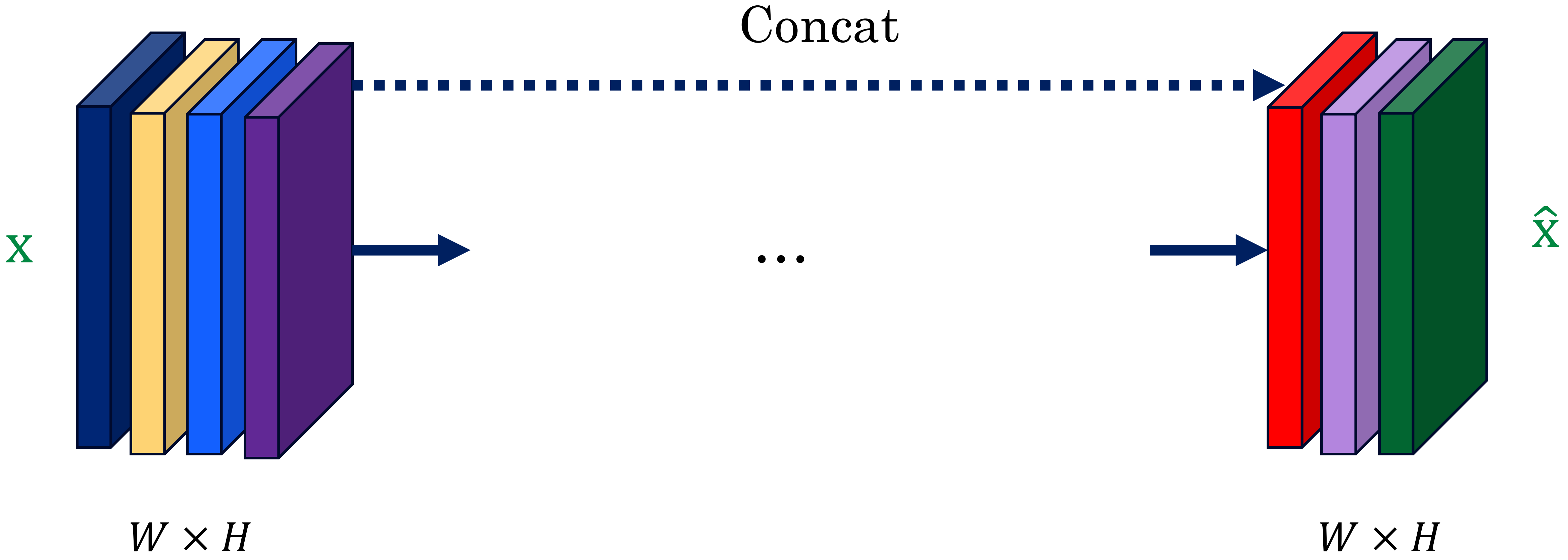
Loss: $\|\hat{x} - x\|_2^2$

Do not edit
How to change the design

What are some shortcomings of traditional CNNs with an information bottleneck?

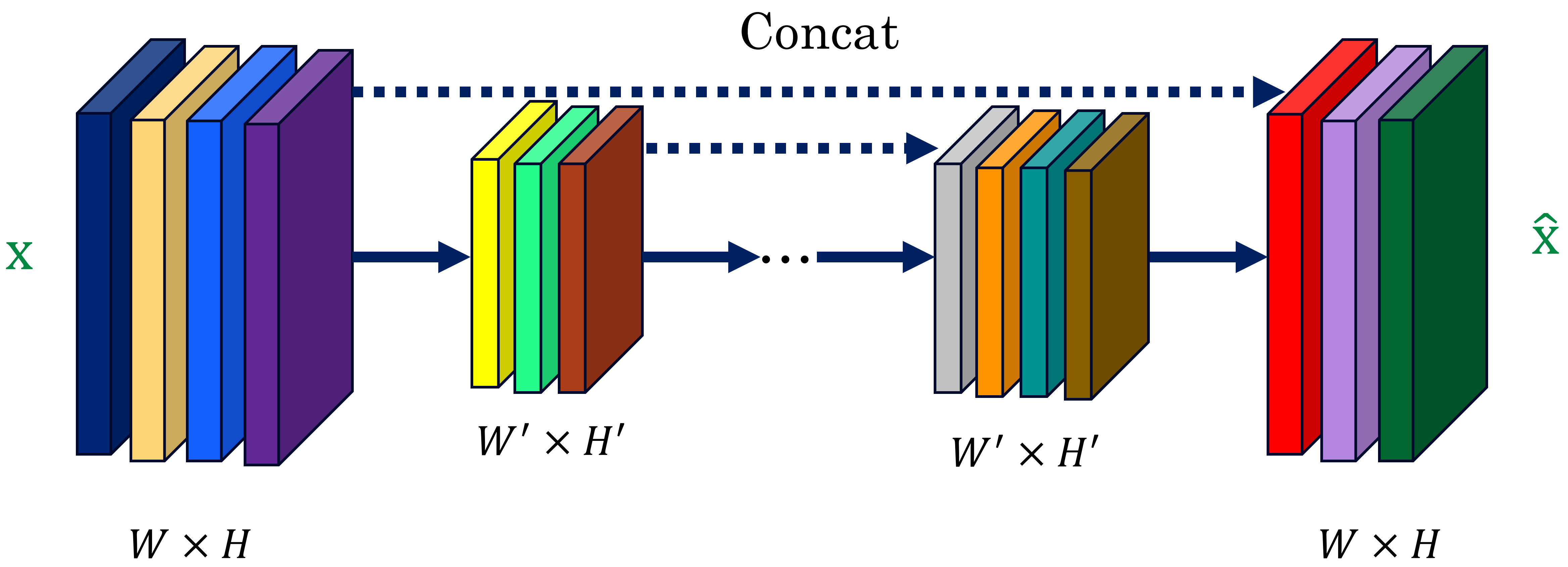
U-Net: Another Encoder-Decoder Architecture

2474272



U-Net: Another Encoder-Decoder Architecture

2474272

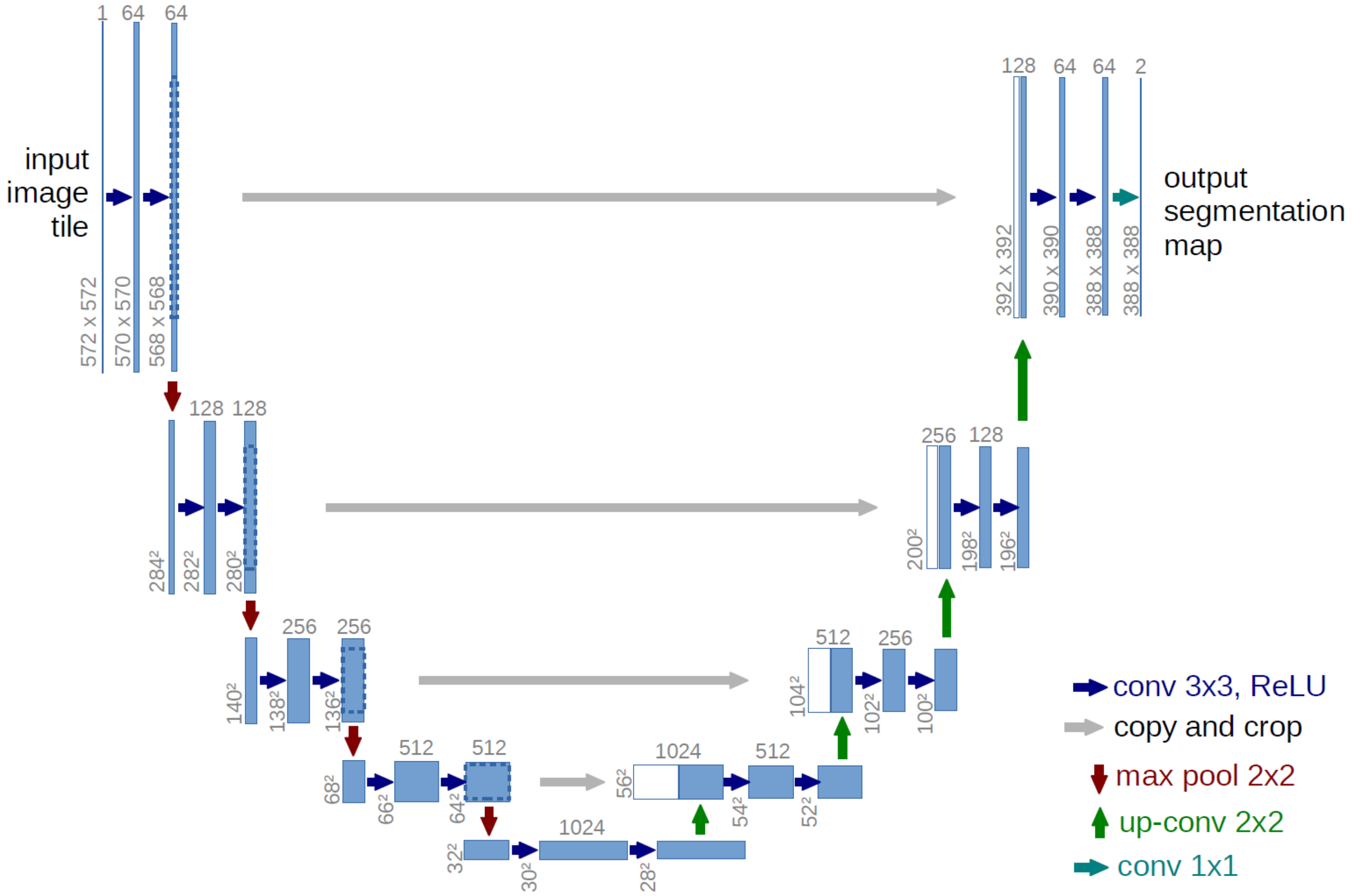


2474272

U-Net: Another Encoder-Decoder Architecture

https://arxiv.org/pdf/1505.04597

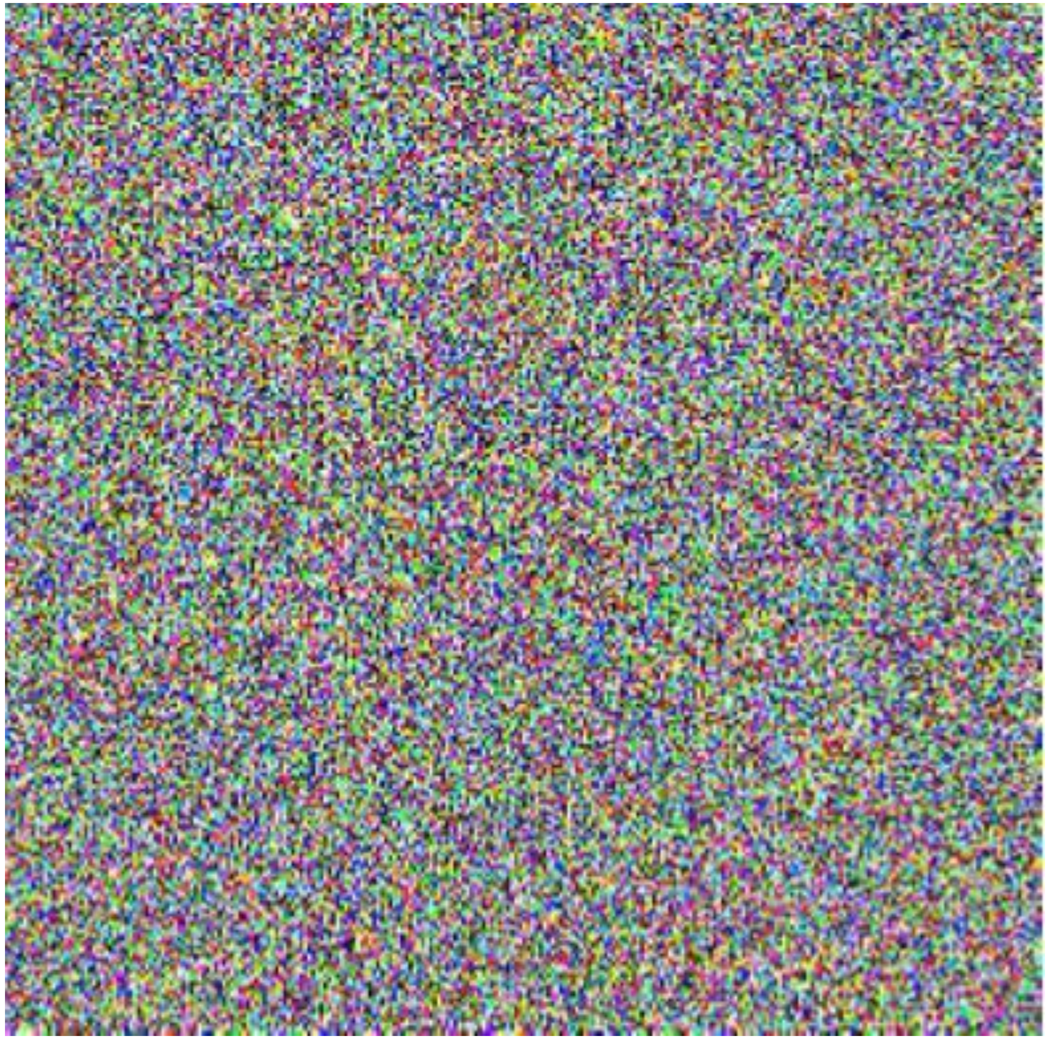
Published in MICCAI (Medical Image Computing and Computer Assisted Interventions) 2015

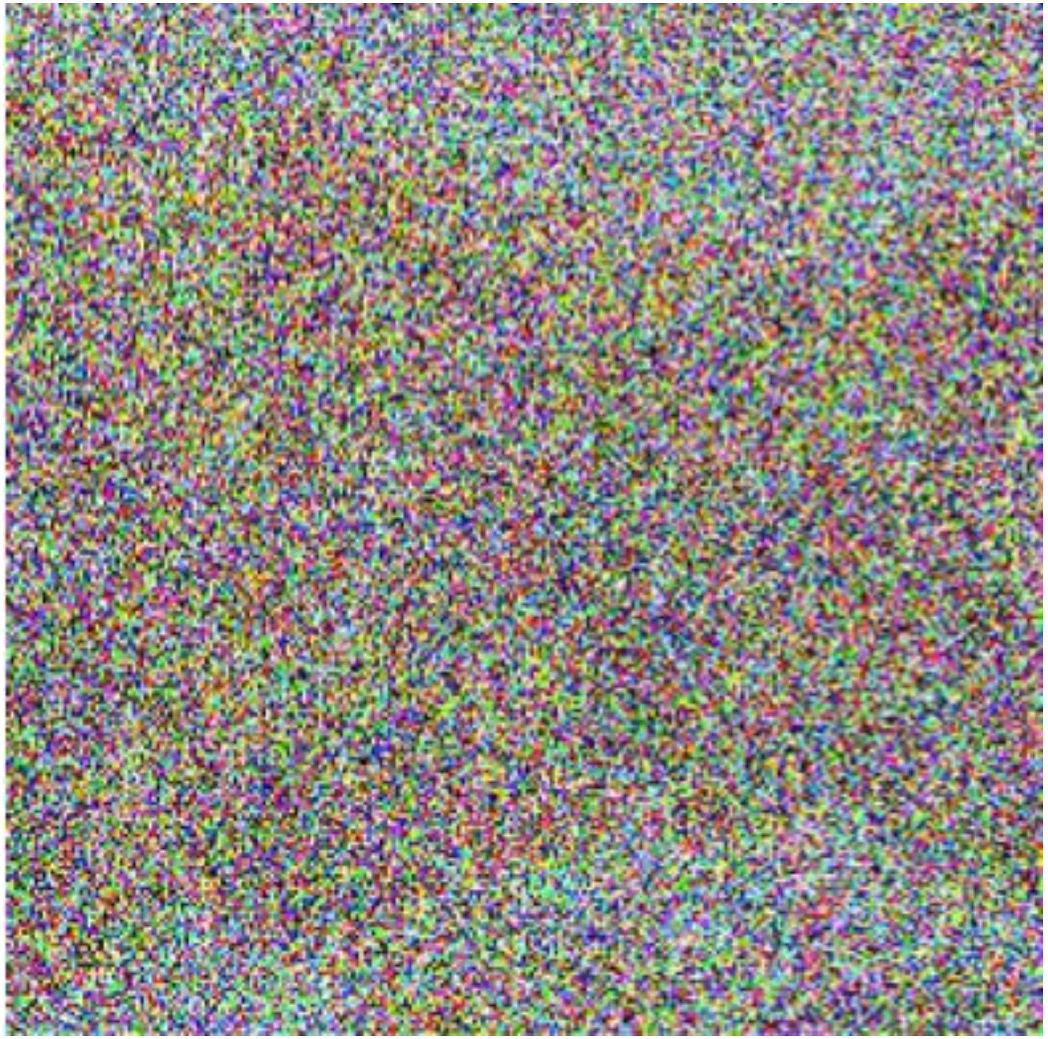


Back to Denoising

From Noise to an Image

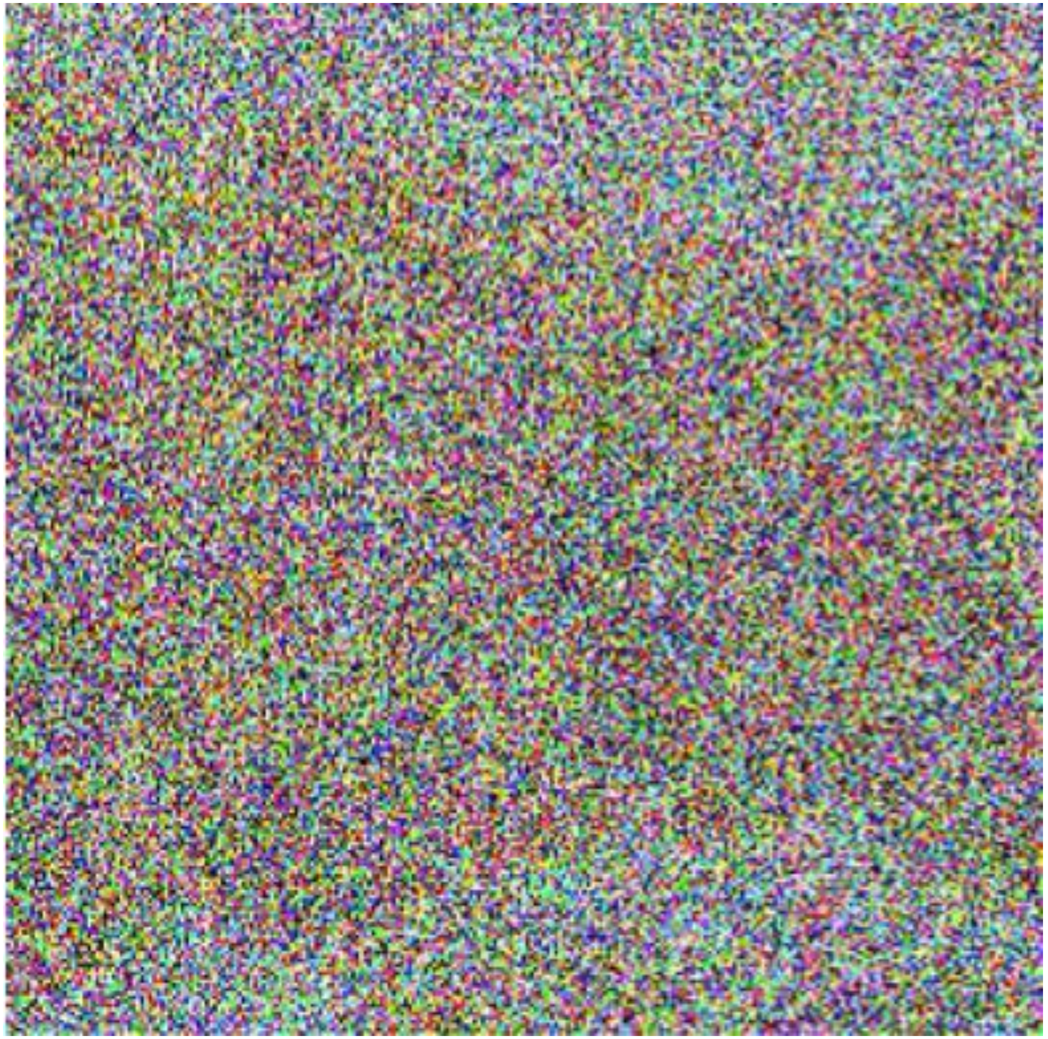
2474272

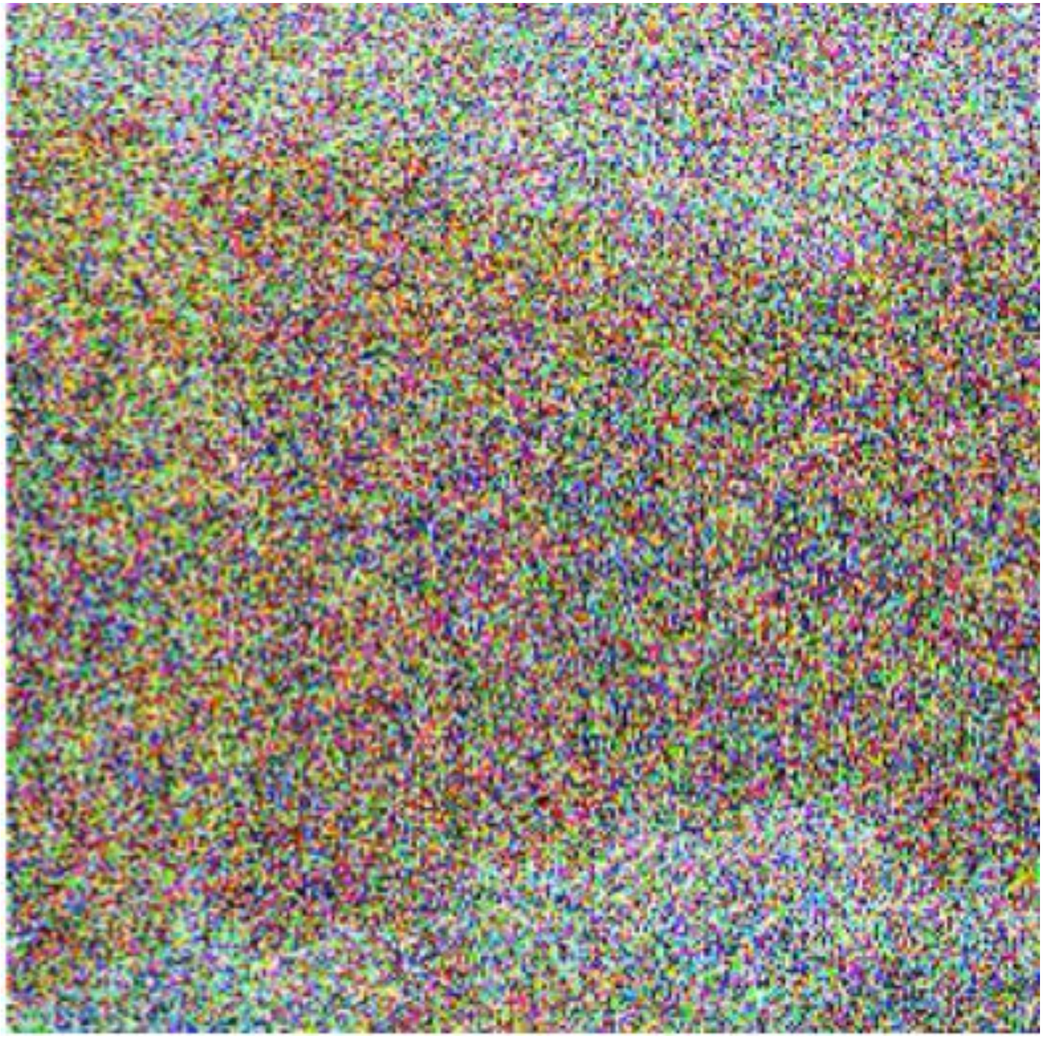




From Noise to an Image

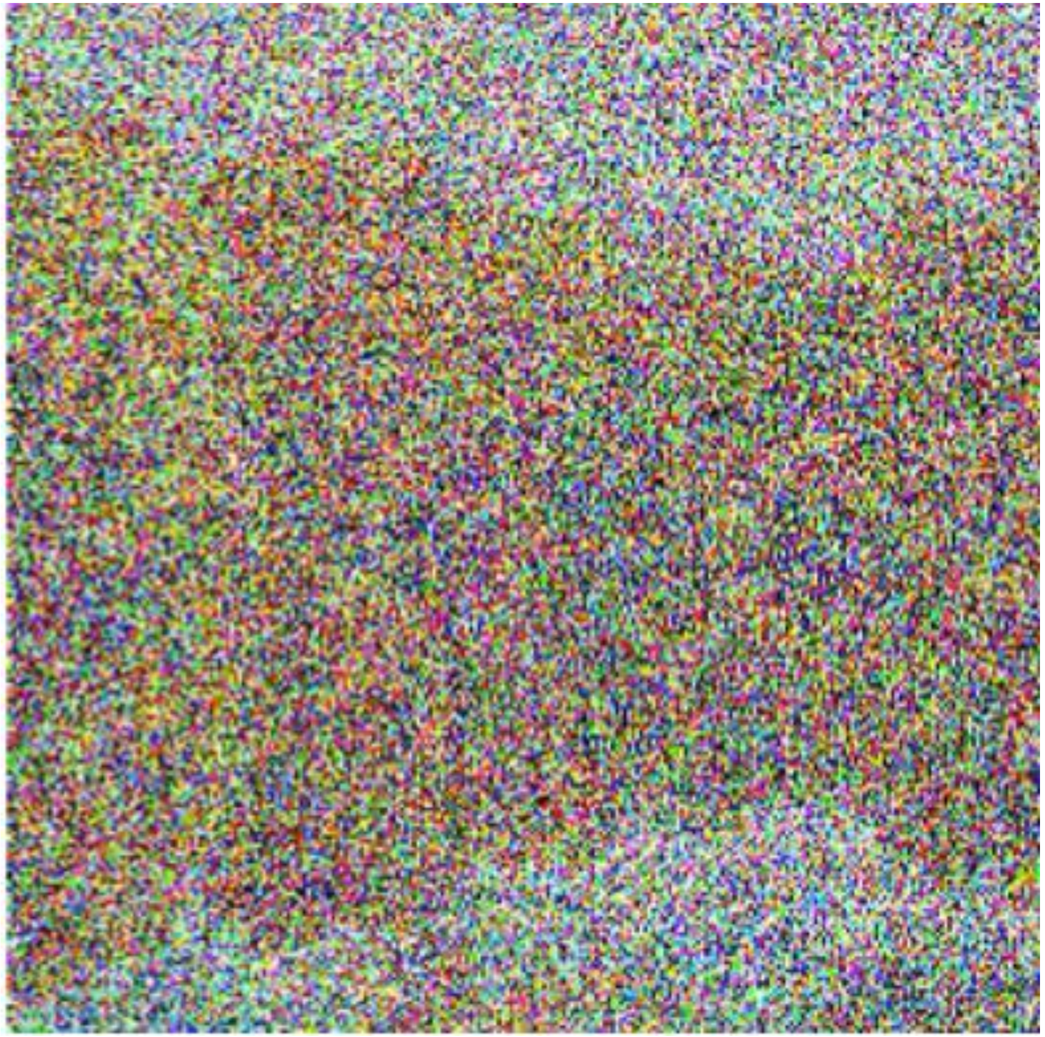
2474272

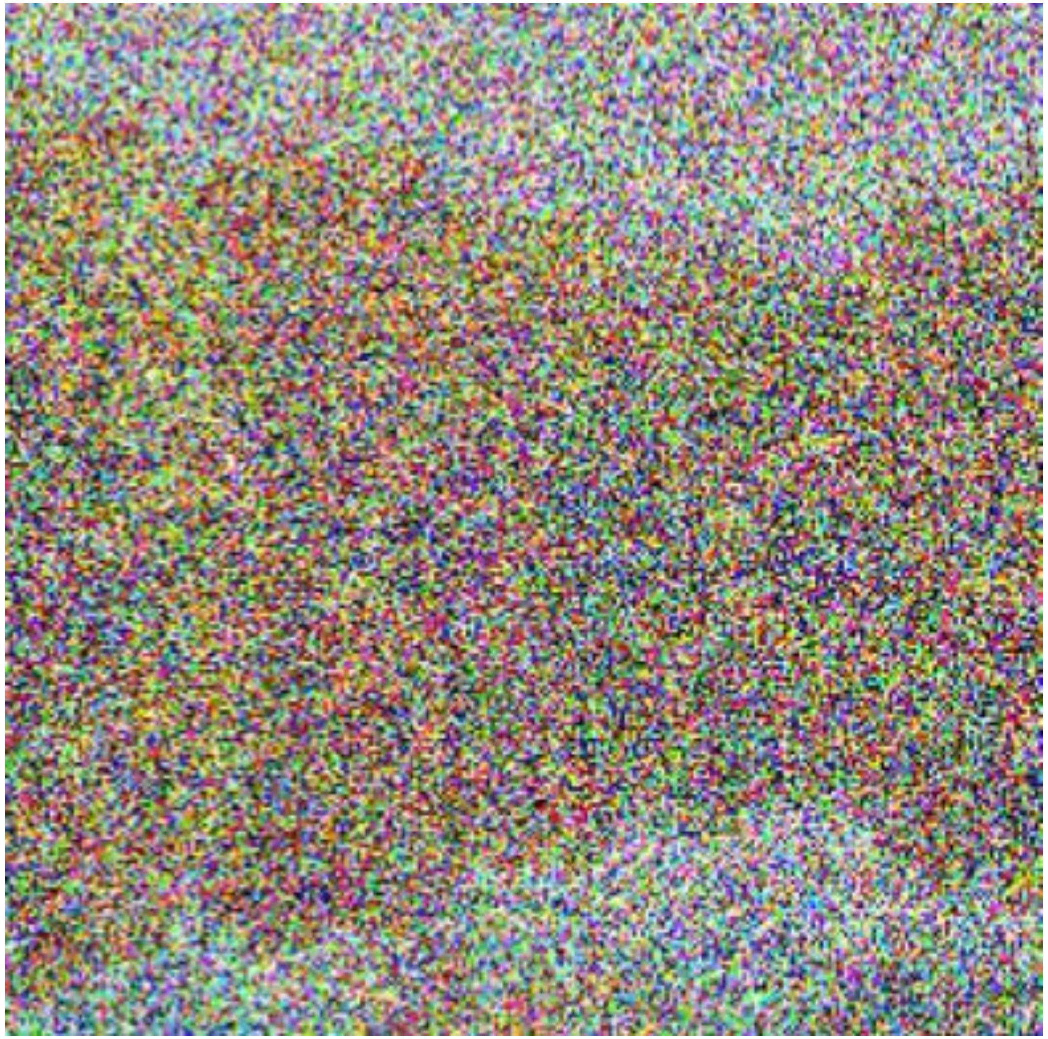




From Noise to an Image

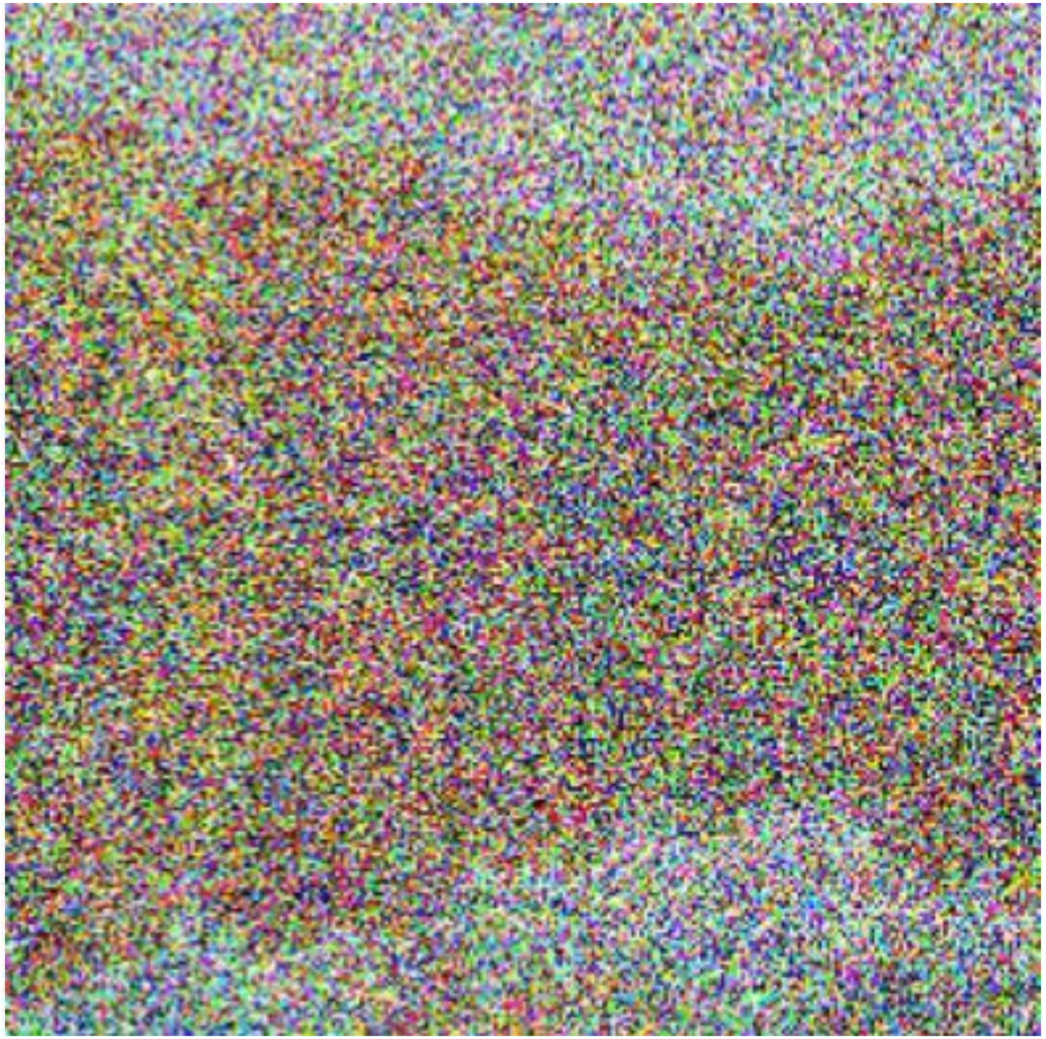
2474272

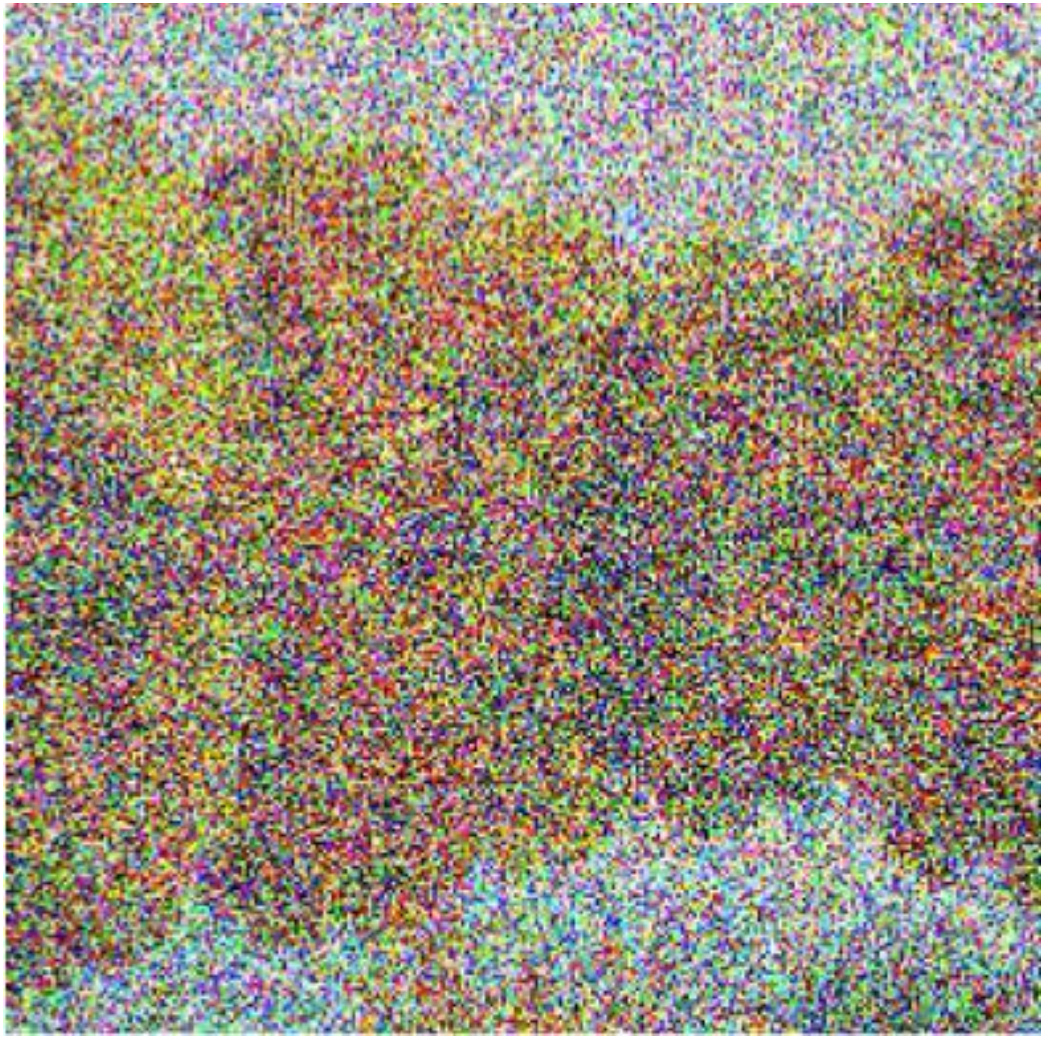




From Noise to an Image

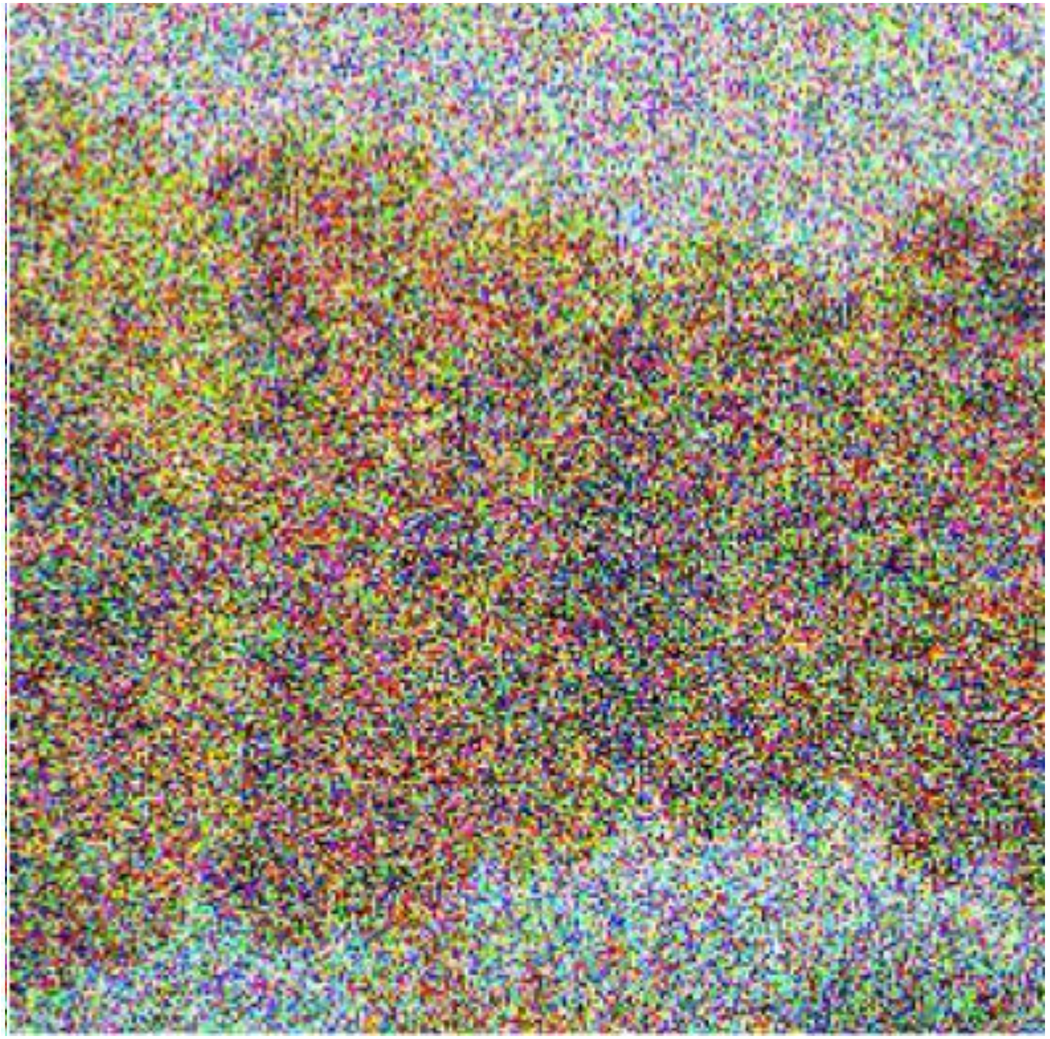
2474272

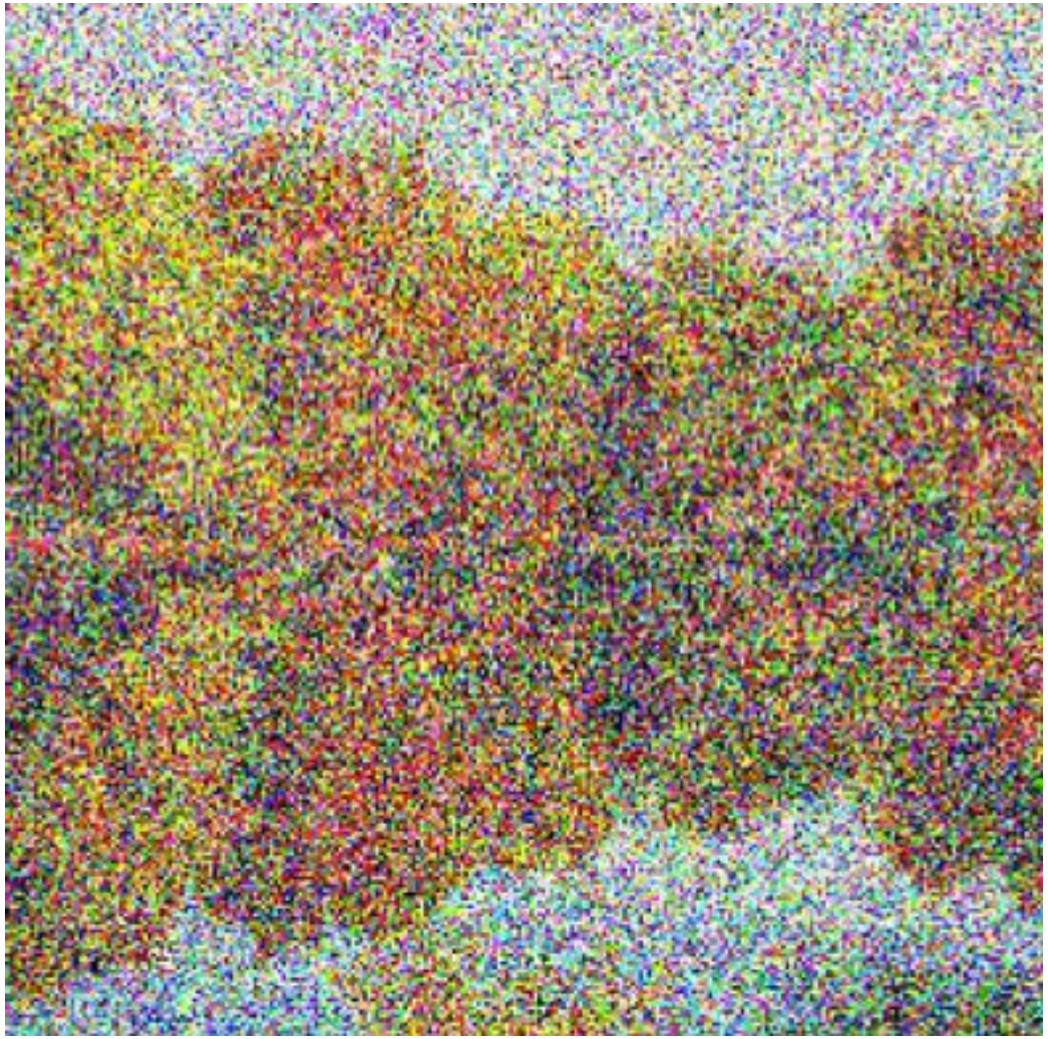




From Noise to an Image

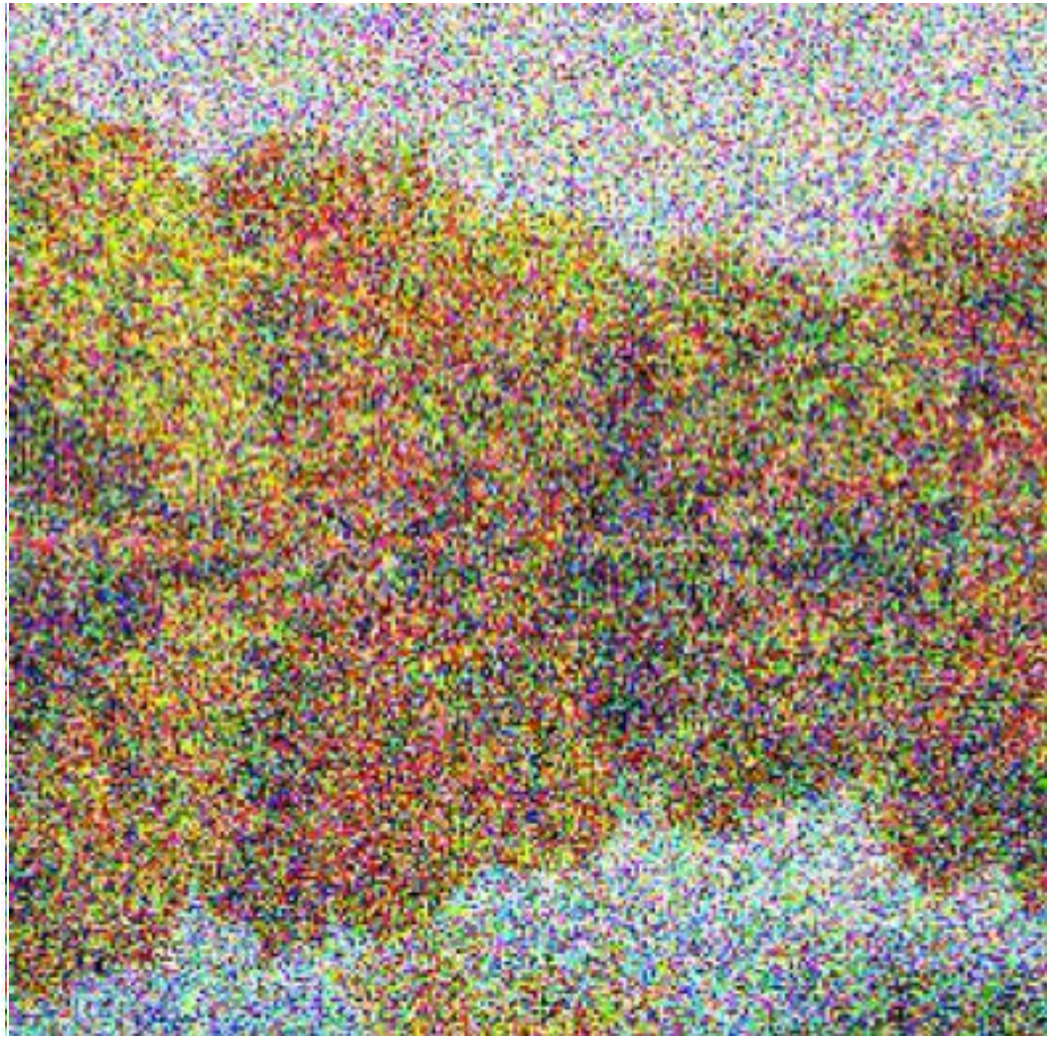
2474272

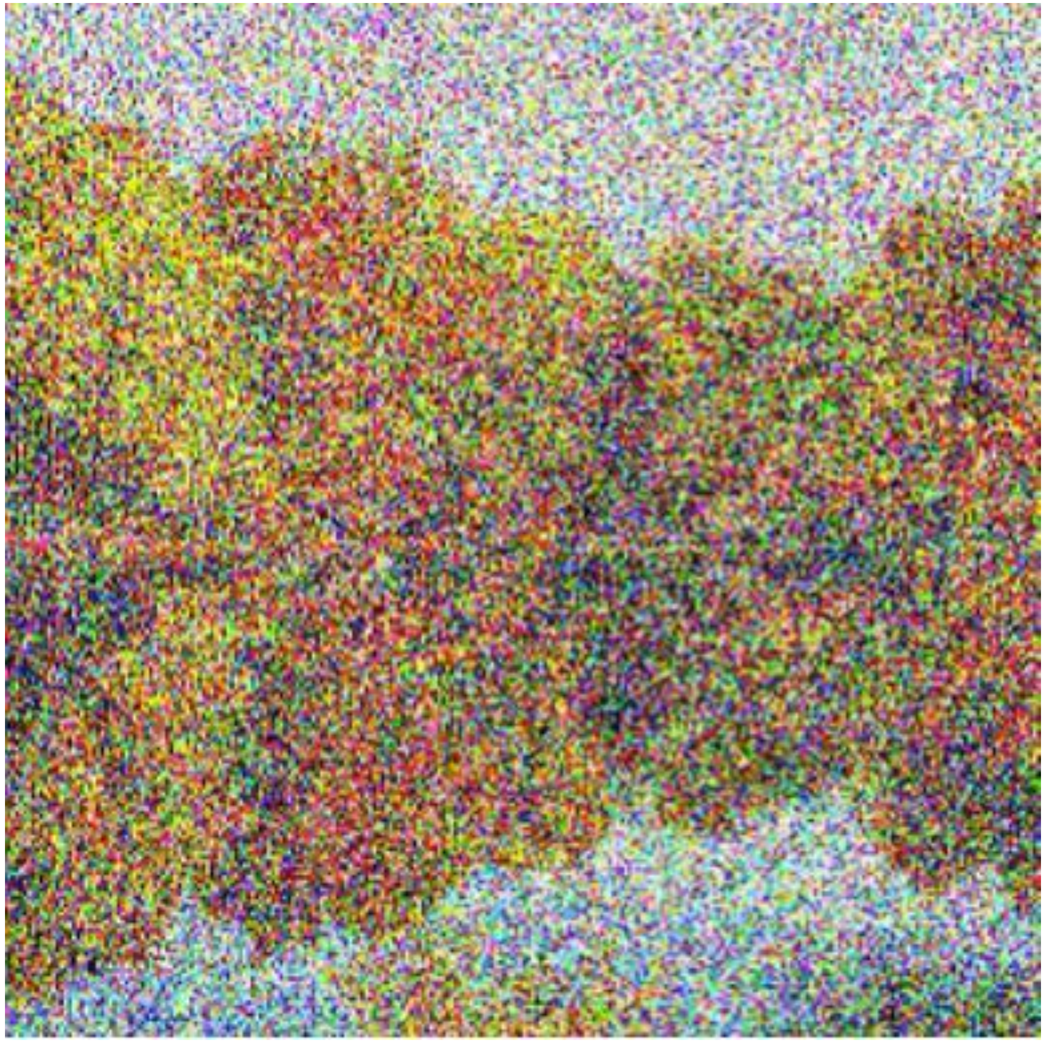




From Noise to an Image

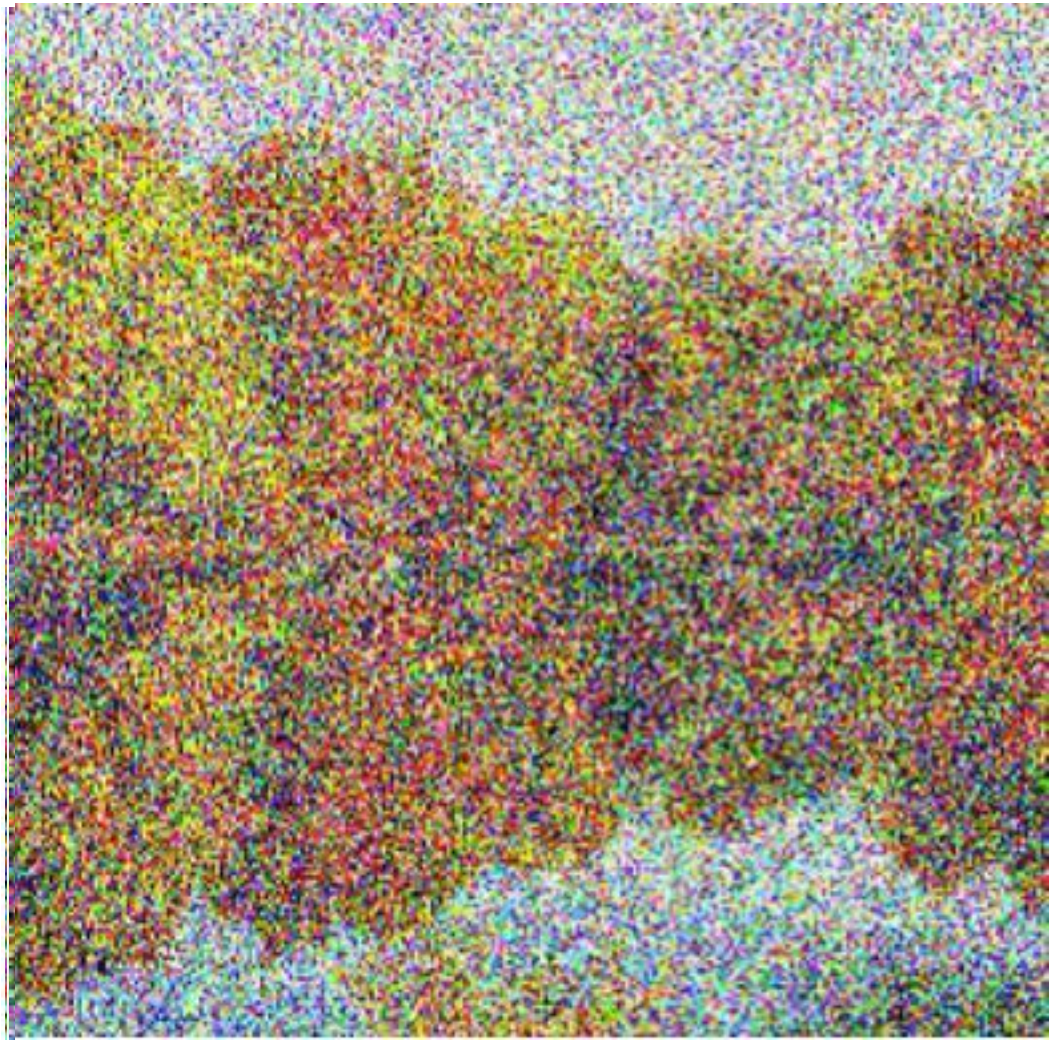
2474272





From Noise to an Image

2474272



From Noise to an Image

2474272

From Noise to an Image

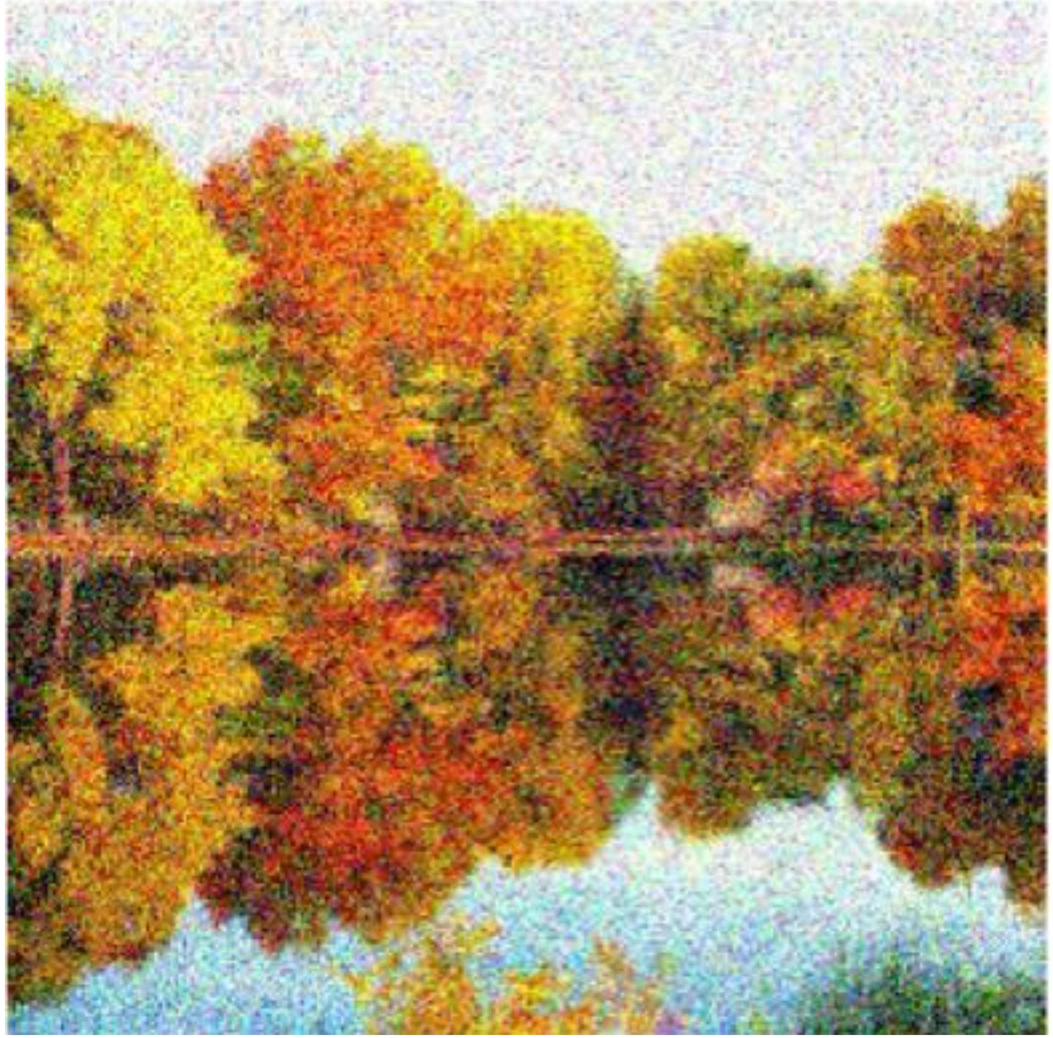
2474272

From Noise to an Image

2474272

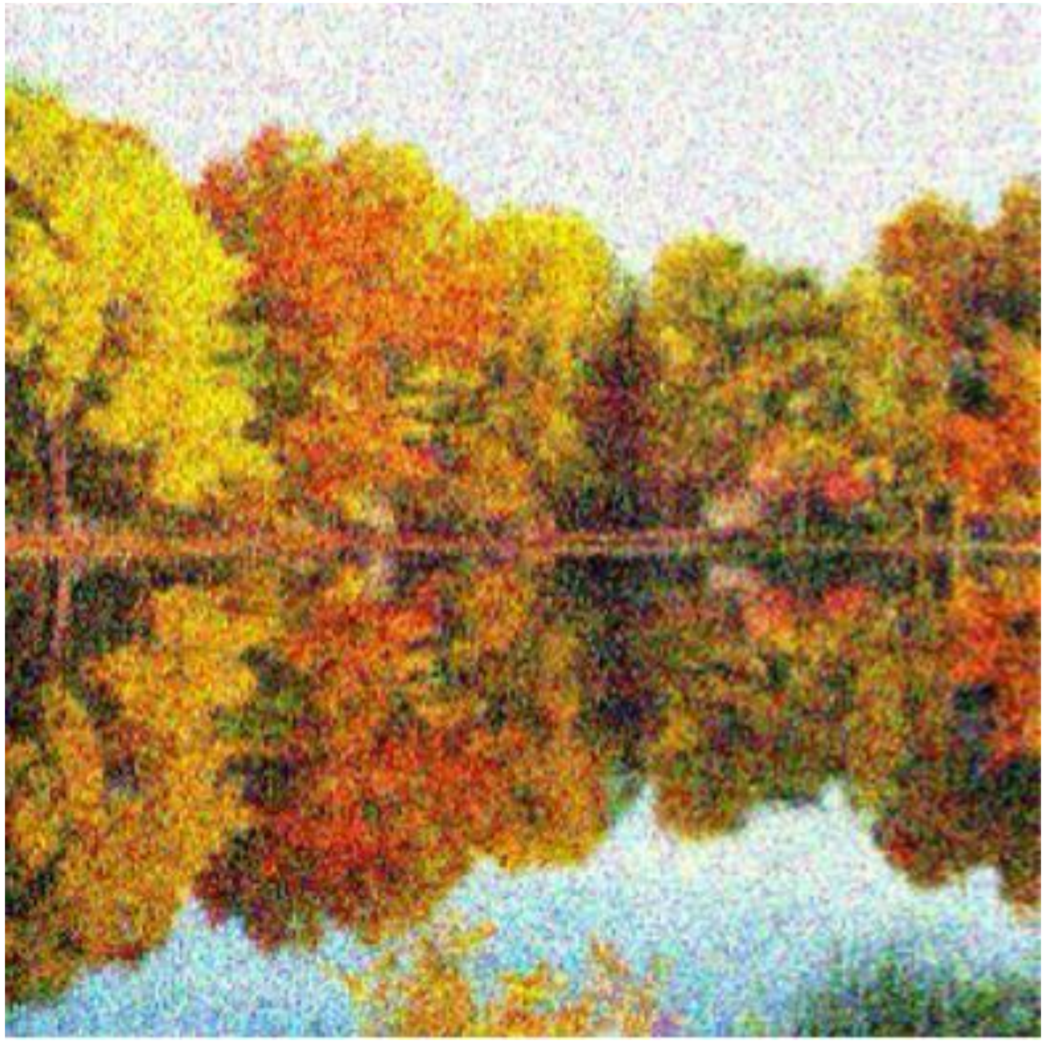
From Noise to an Image

2474272



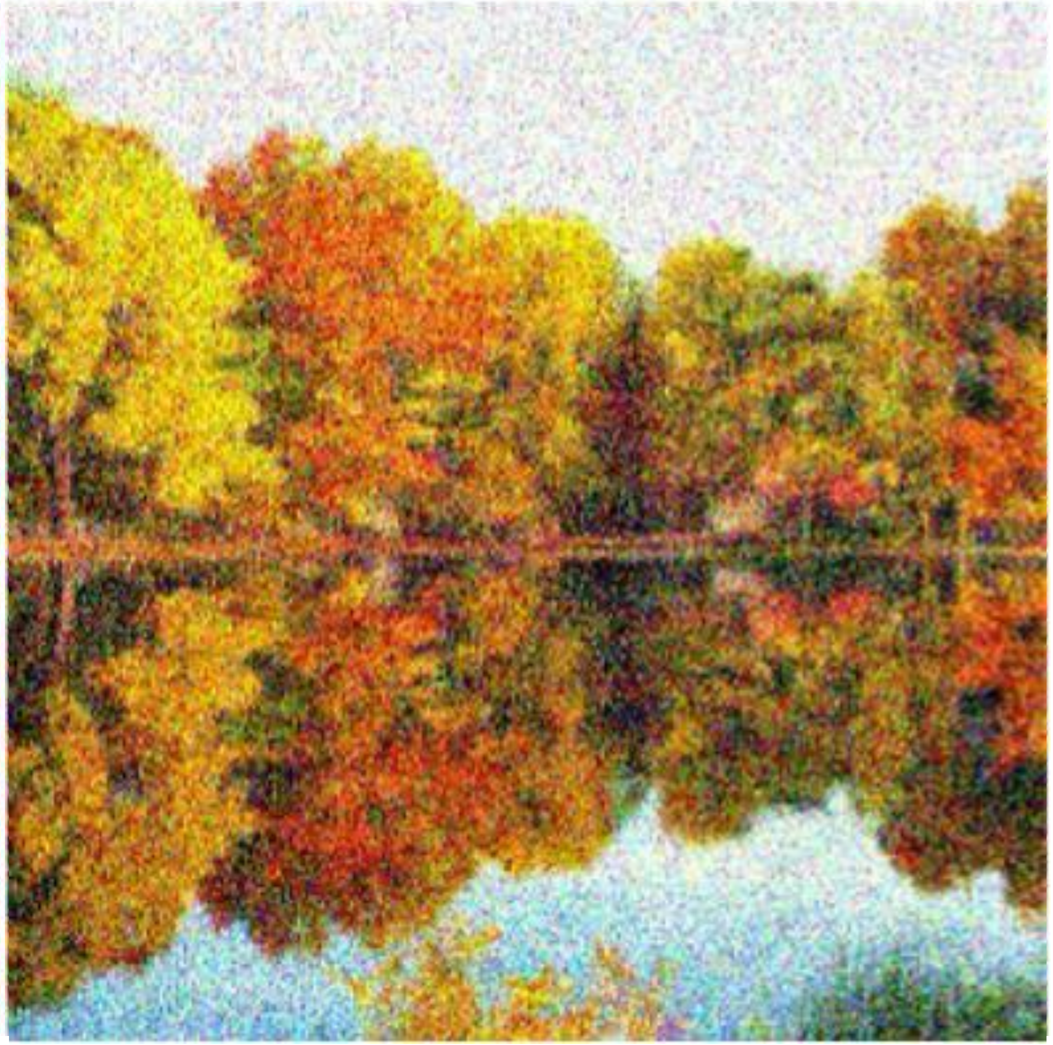
From Noise to an Image

2474272



From Noise to an Image

2474272



From Noise to an Image

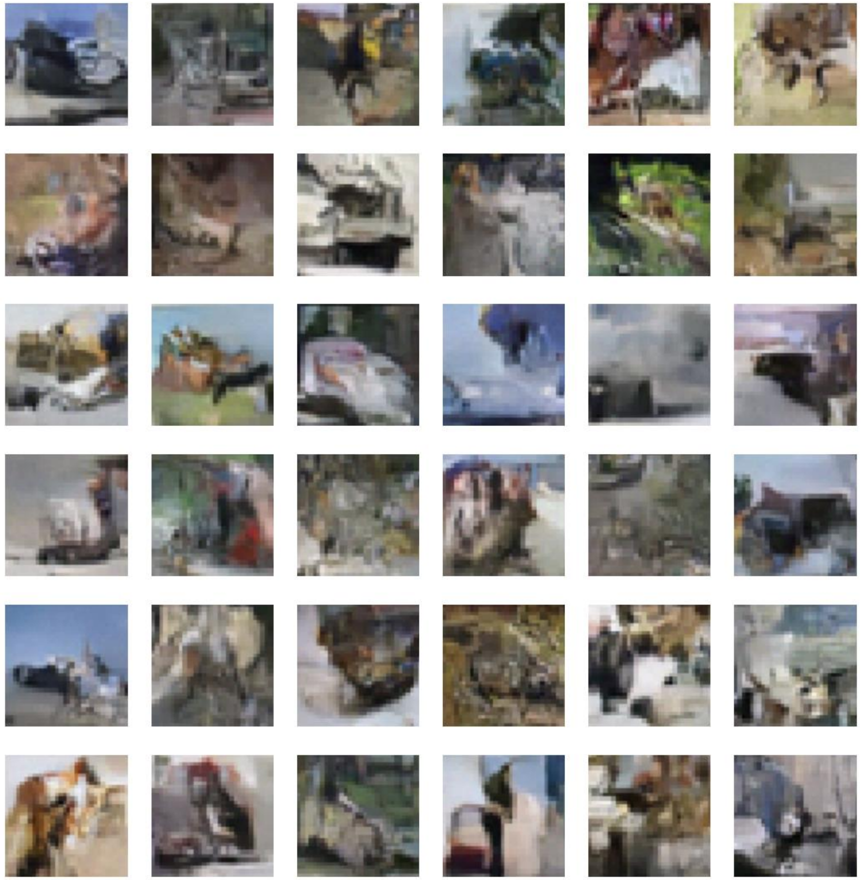
2474272

From Noise to an Image

2474272

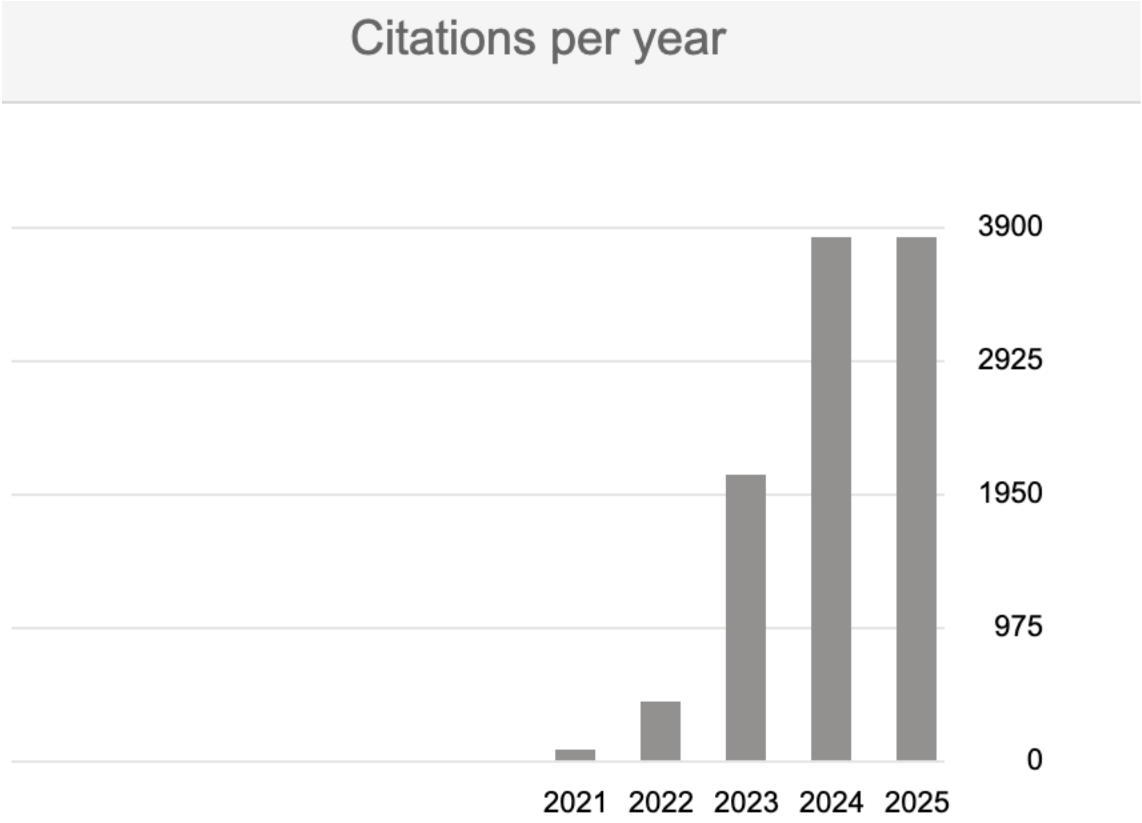
Diffusion Models

2474272



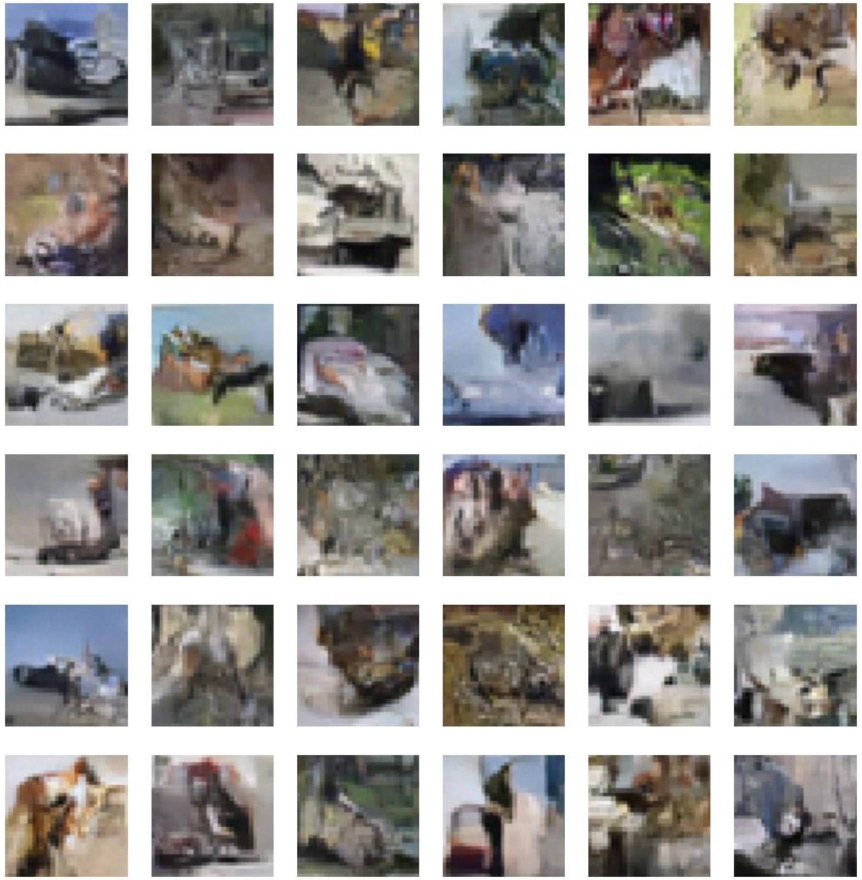
[Sohl-Dickstein et al., 2015]

[Deep unsupervised learning using nonequilibrium thermodynamics]()

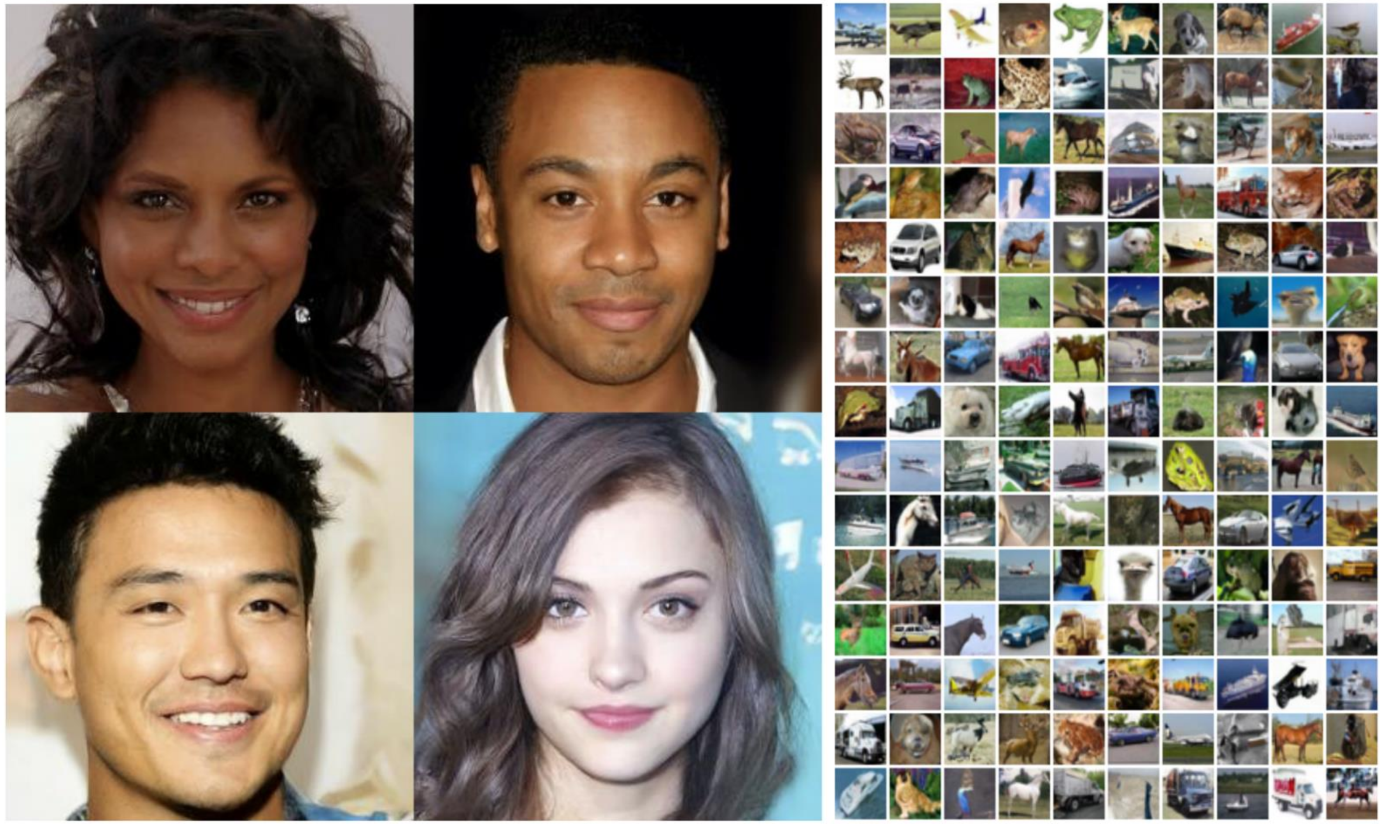


Diffusion Models

2474272



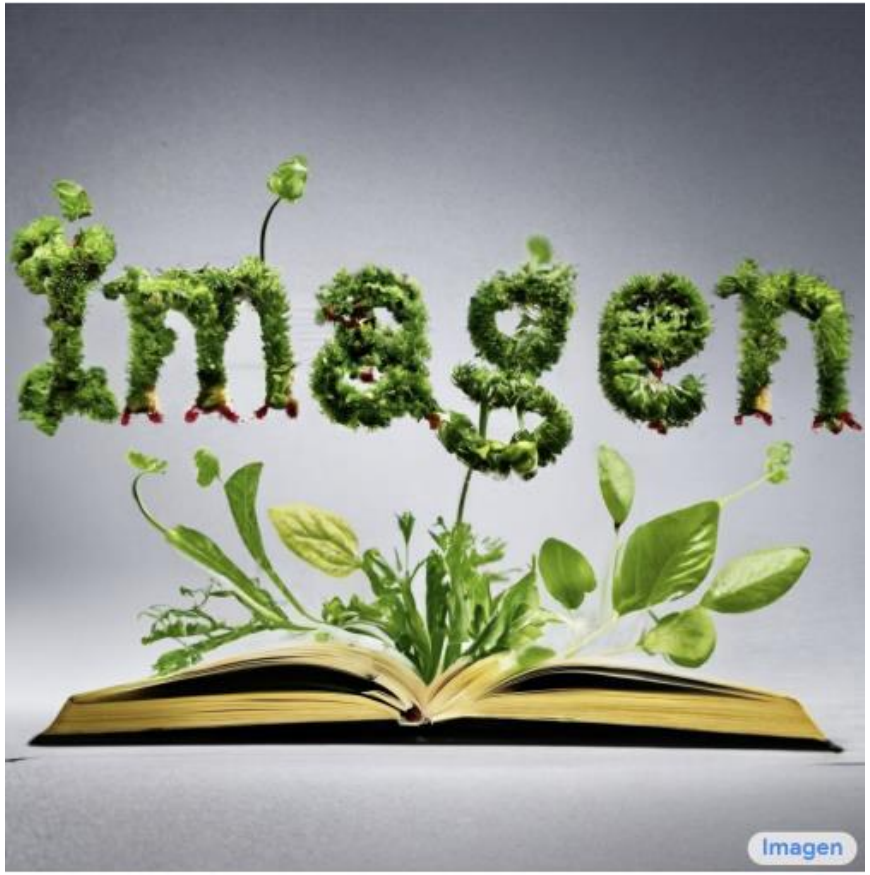
[Sohl-Dickstein et al., 2015]



[Ho, et al., "Denoising Diffusion", 2020]

Imagen

2474272



Sprouts in the shape of text 'Imagen' coming out of a fairytale book.

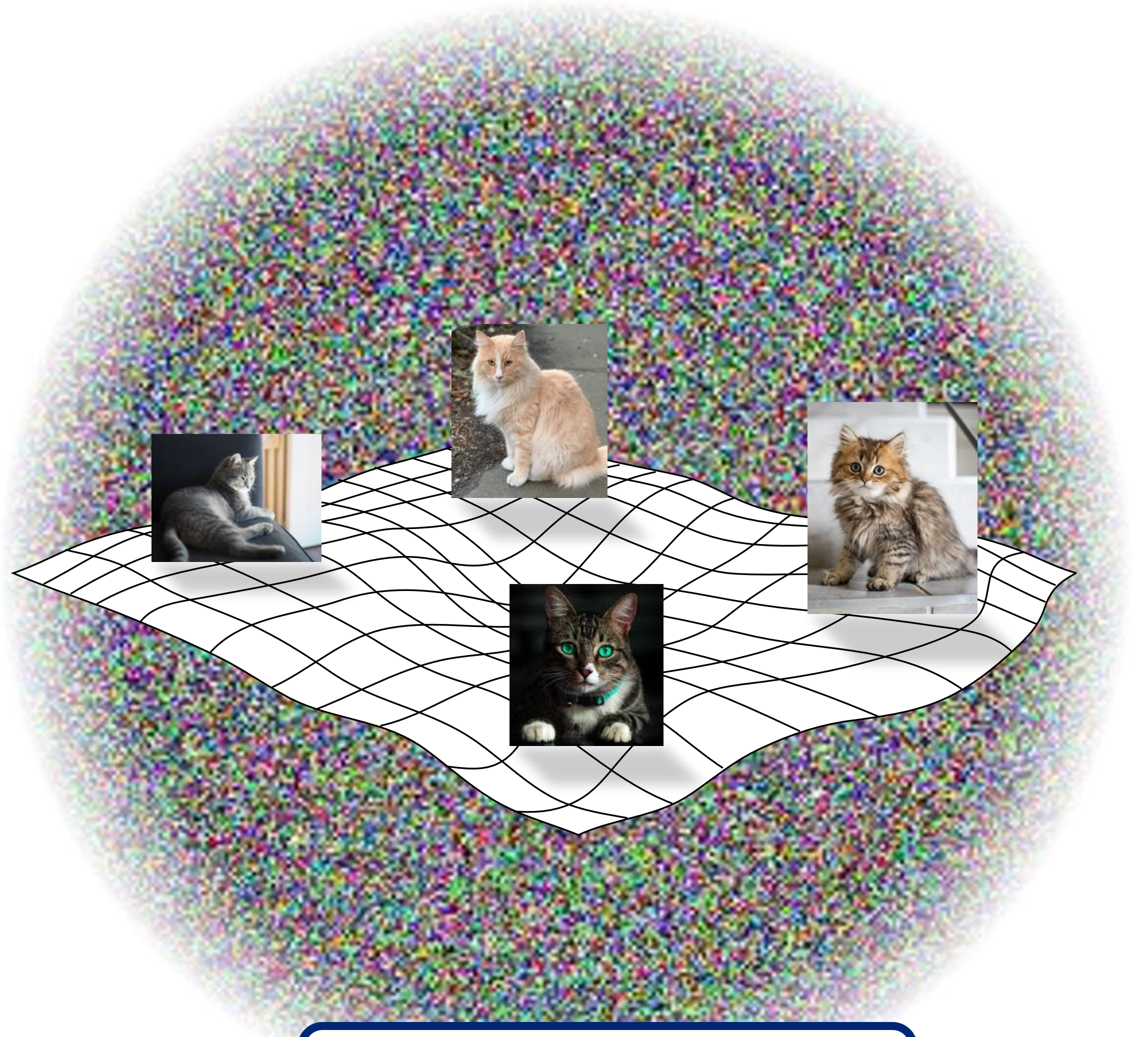
A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.

[Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding]()

Recall: Natural Image Manifolds

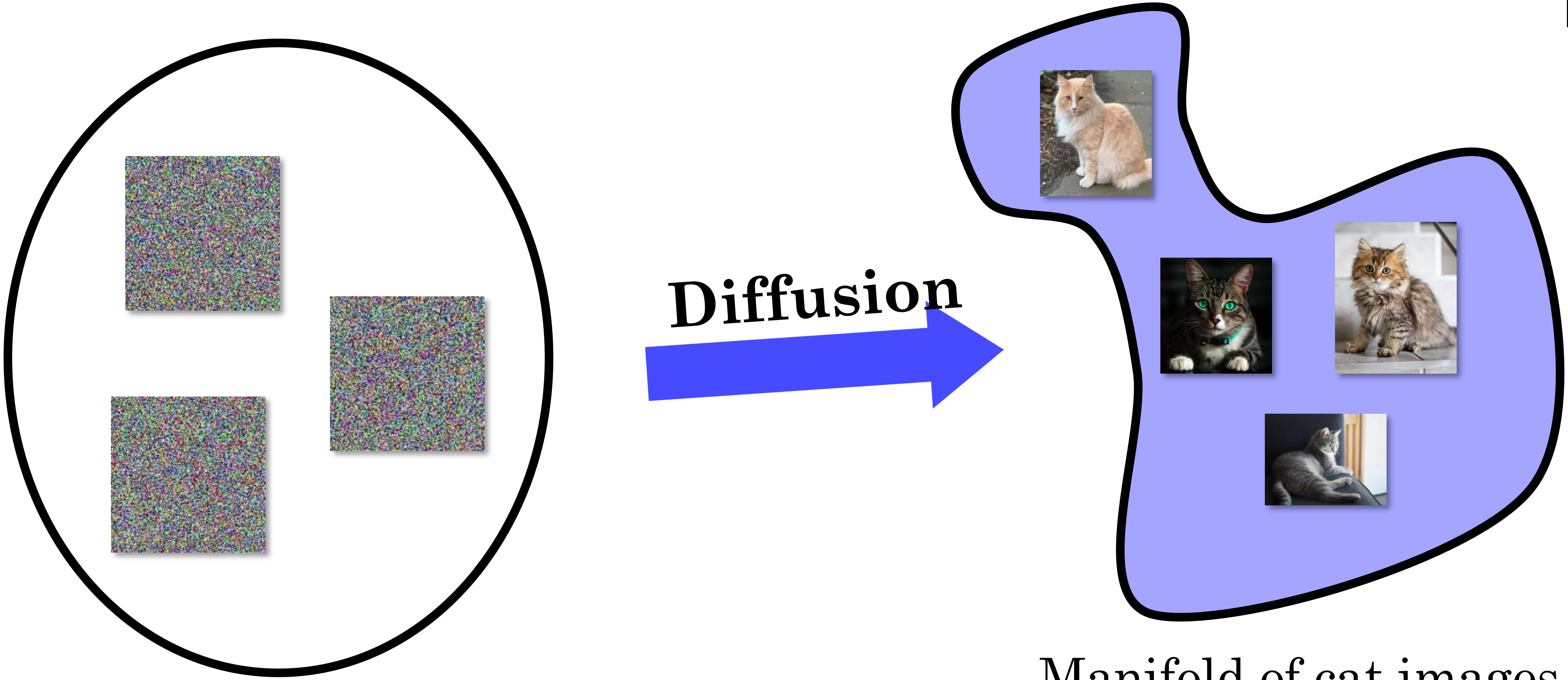
2474272

- Most images are "noise".
- "Meaningful" images tend to form some manifold within the space of all images.
- Images of a particular class fall on manifolds within that manifold…



The Space of All Images

2474272

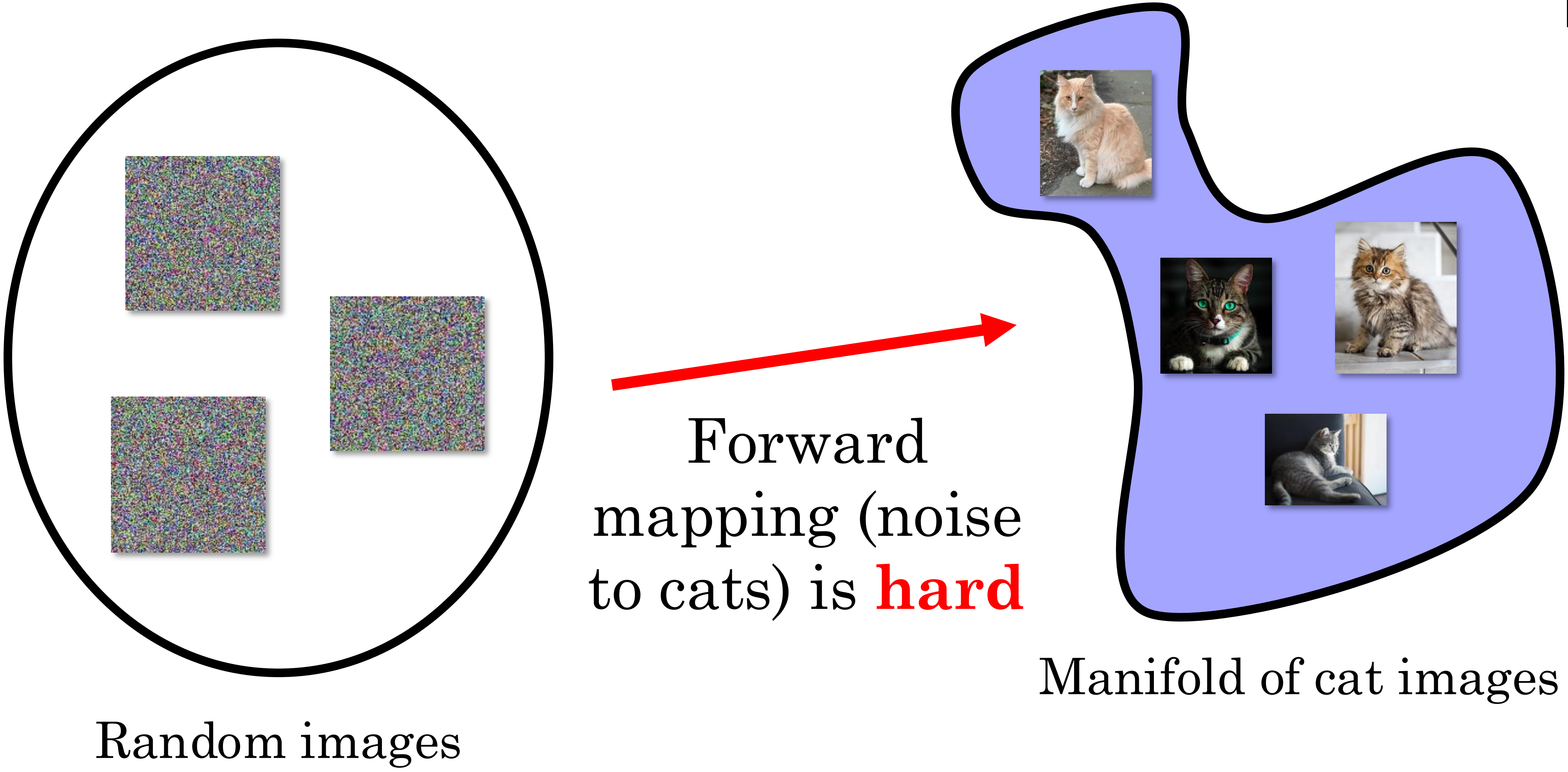


Random images

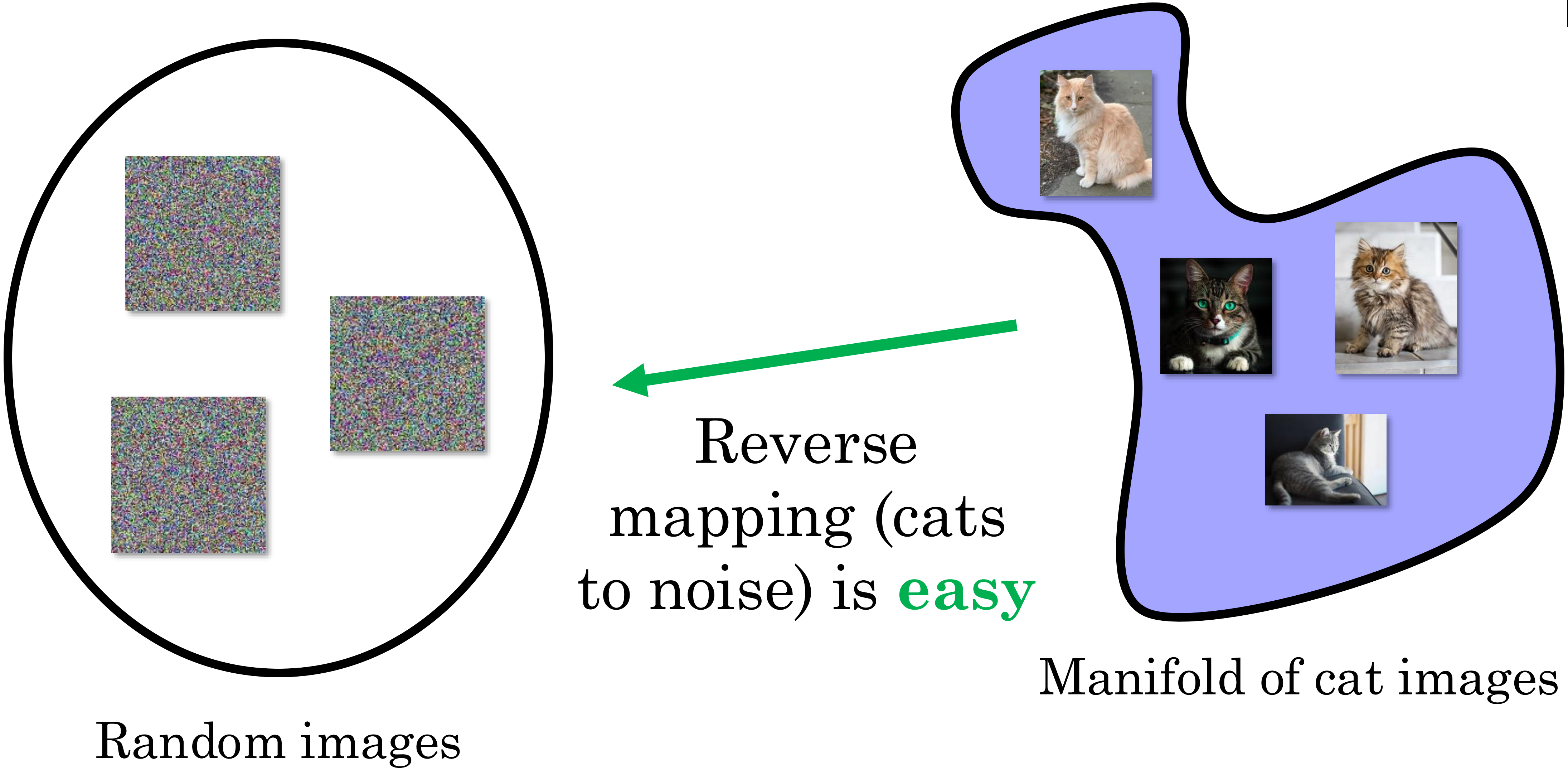
Manifold of cat images

Slide concept: Steve Seitz

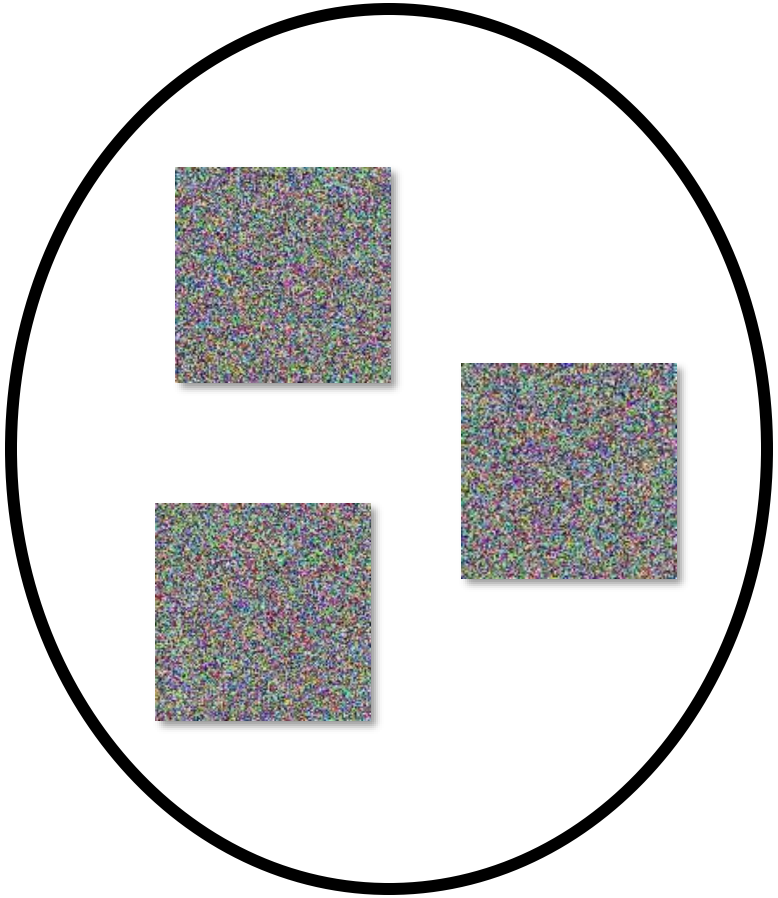
2474272



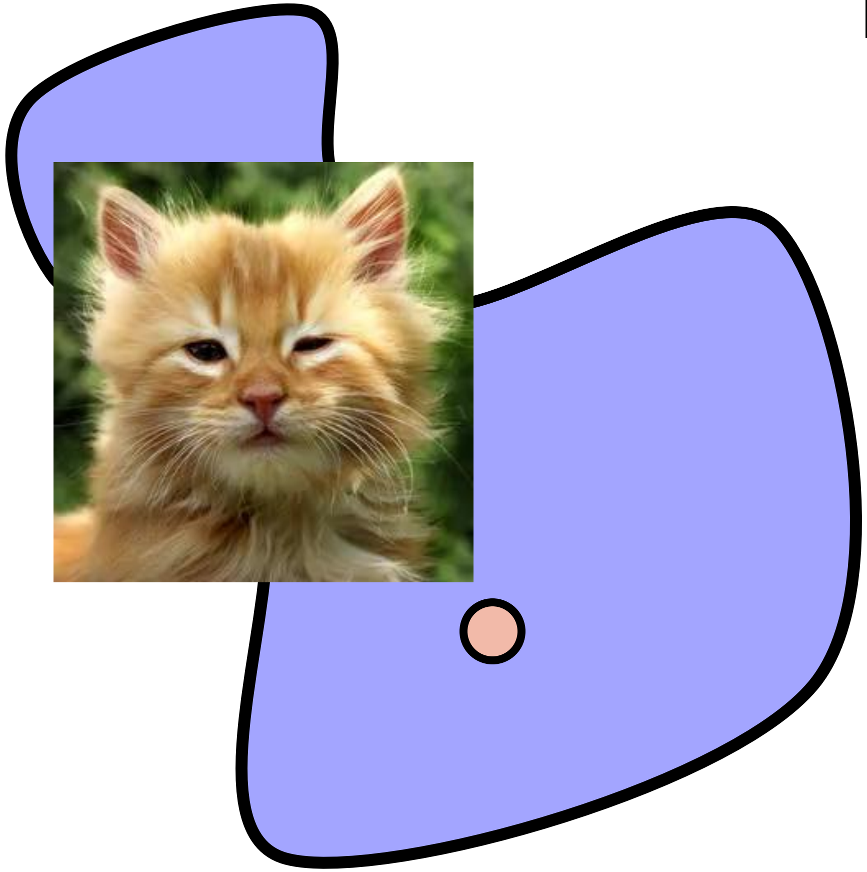
Slide concept: Steve Seitz


Reverse mapping (cats to noise) is **easy**
Manifold of cat images
Random images

2474272

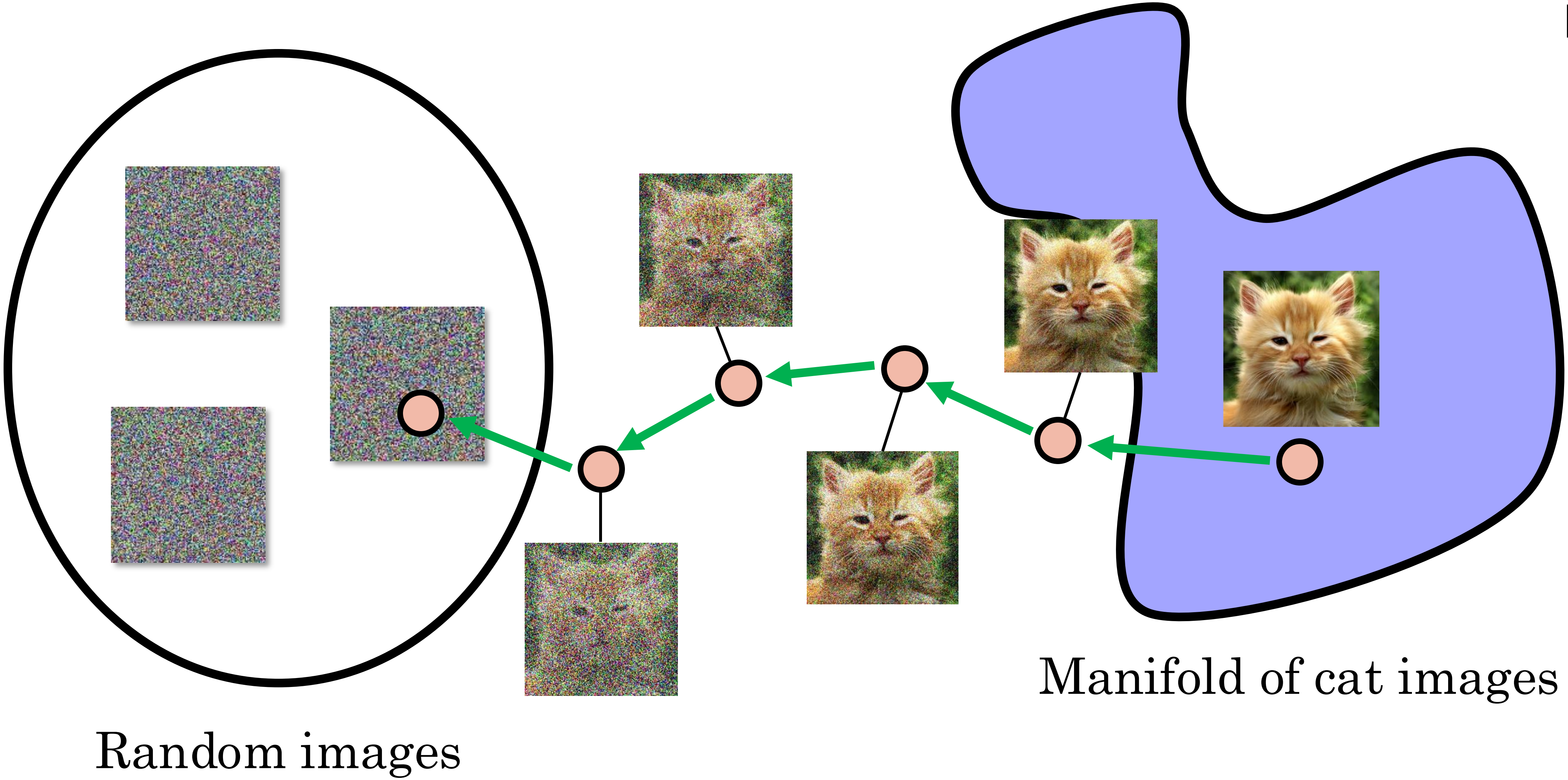


Random images



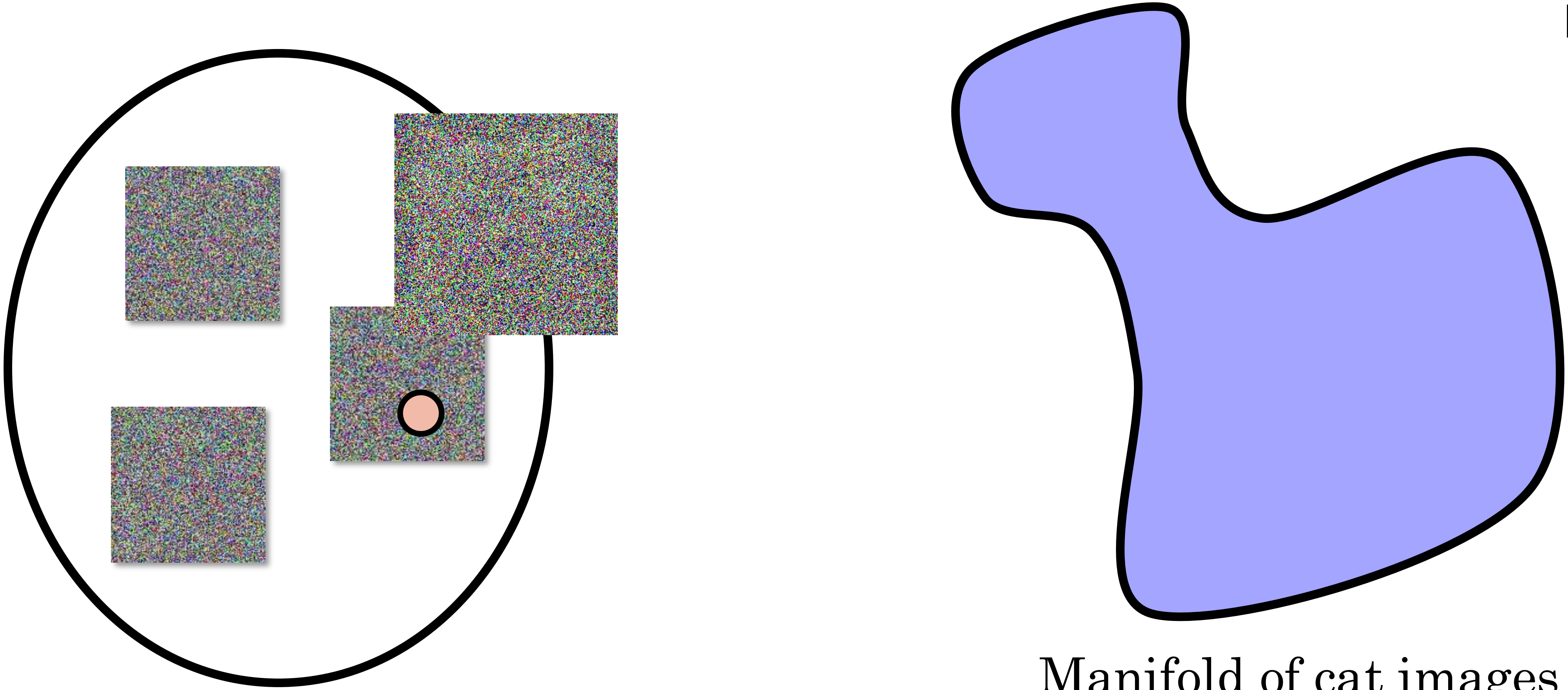
Manifold of cat images

Slide concept: Steve Seitz



2474272

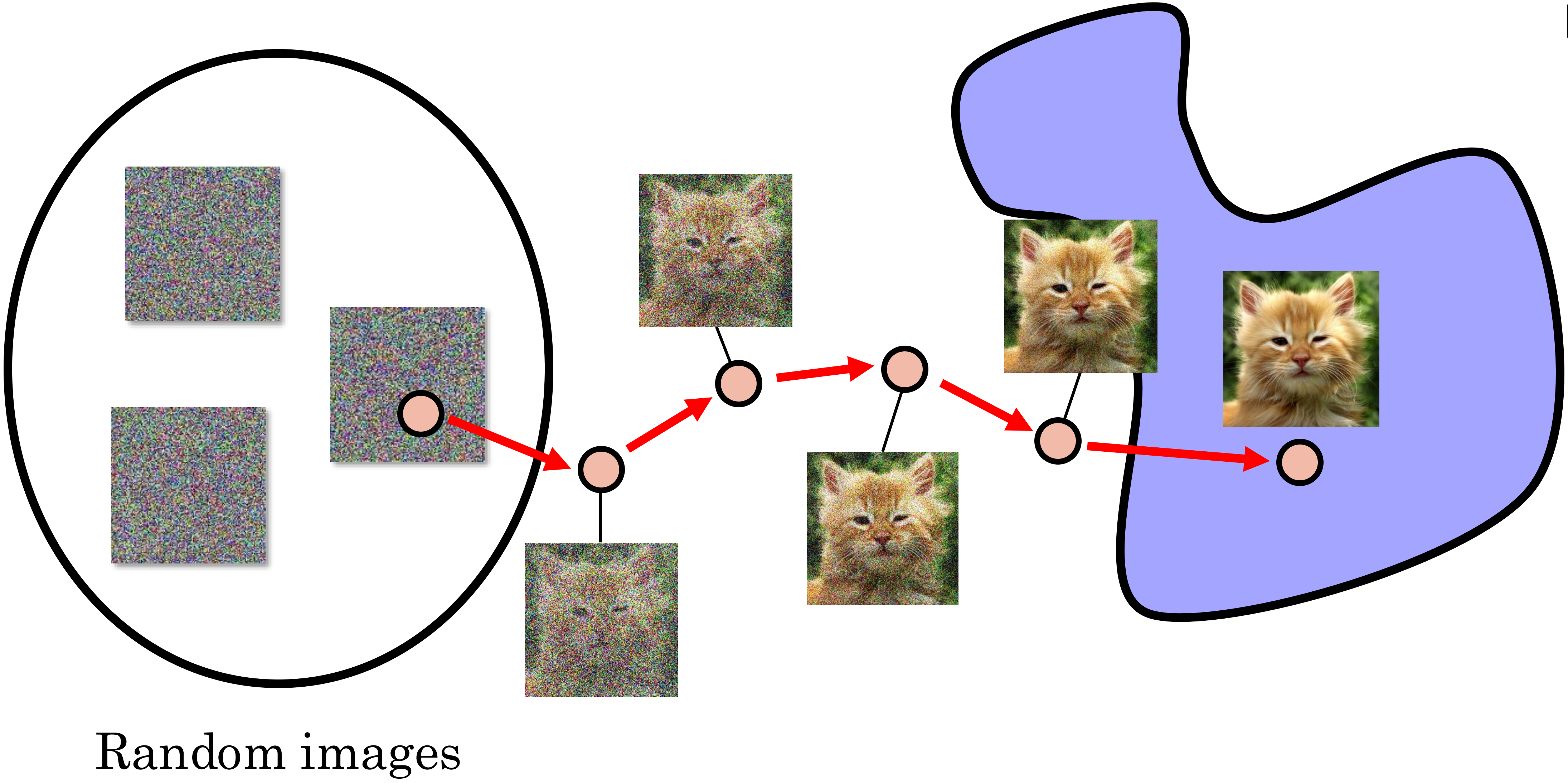
Slide concept: Steve Seitz

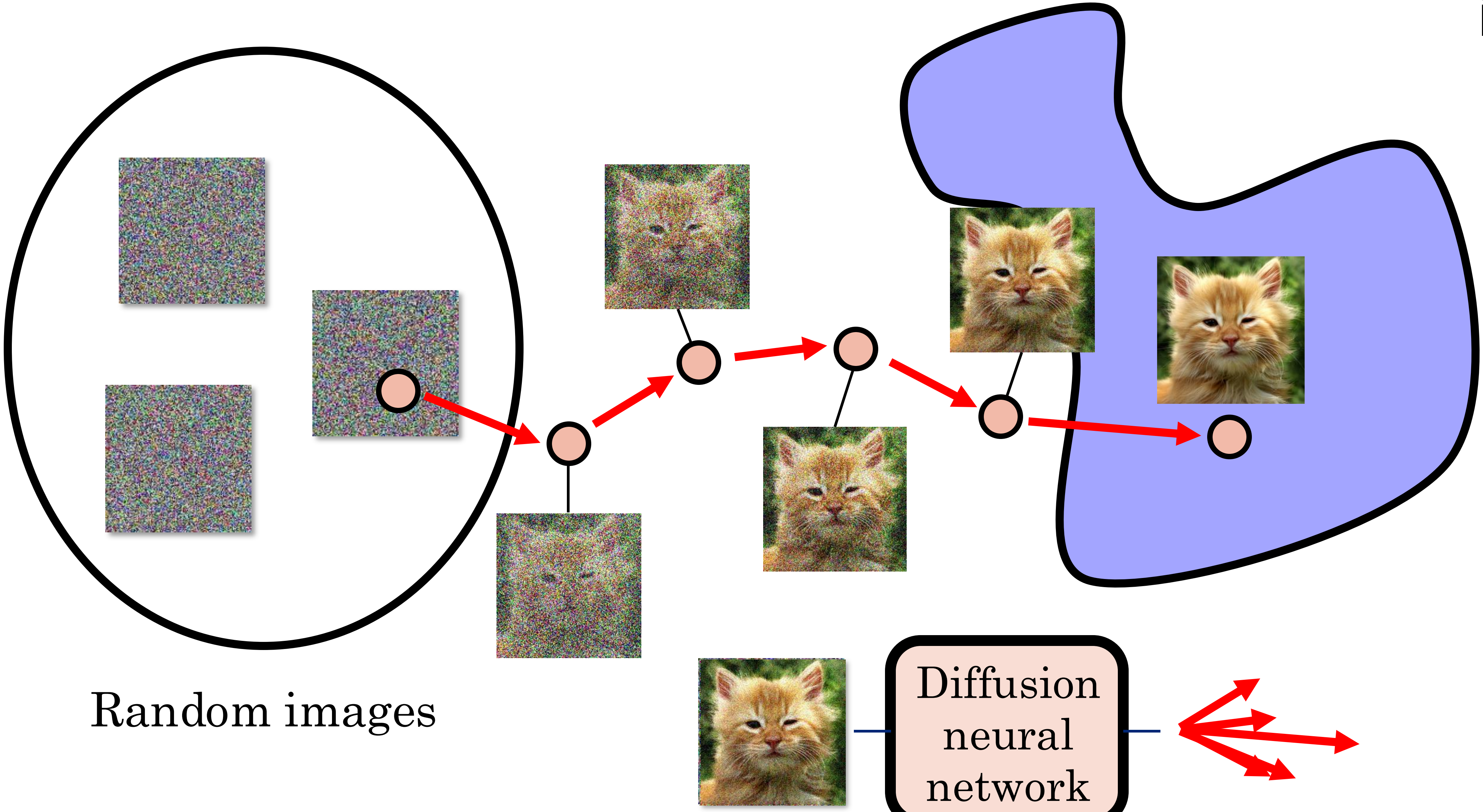


Random images

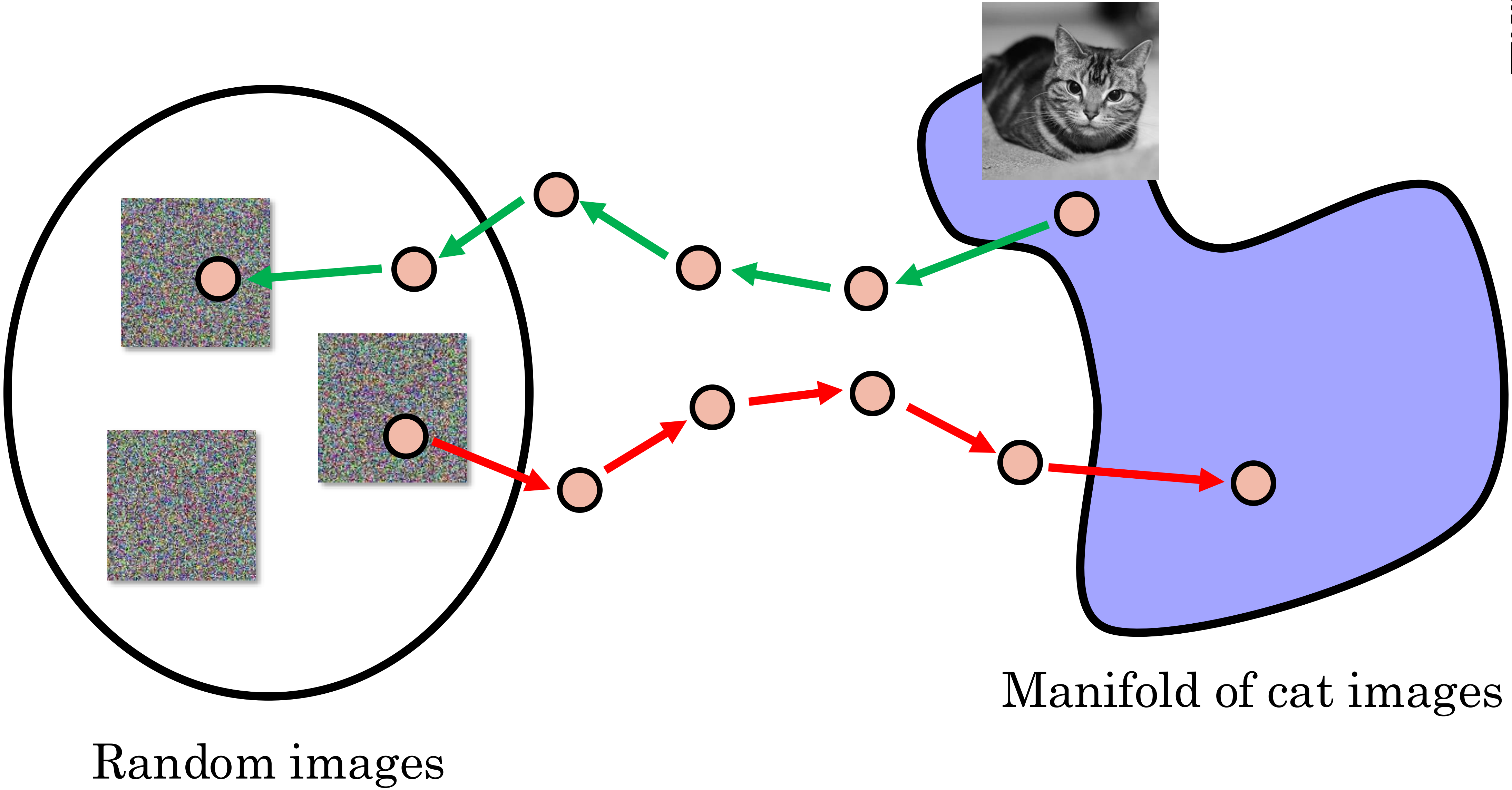
Manifold of cat images

Slide concept: Steve Seitz


Random images

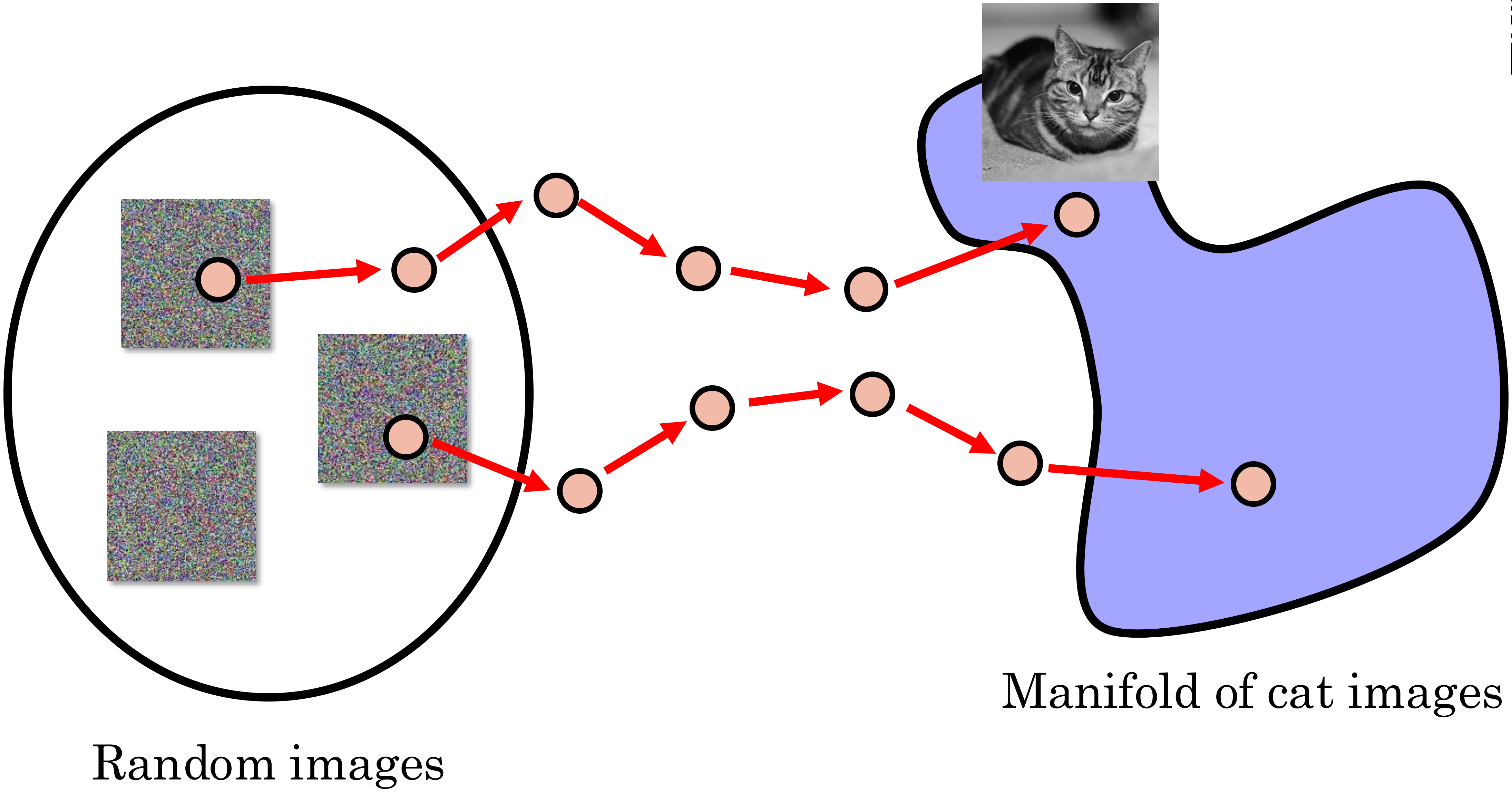


Random images
Diffusion neural network



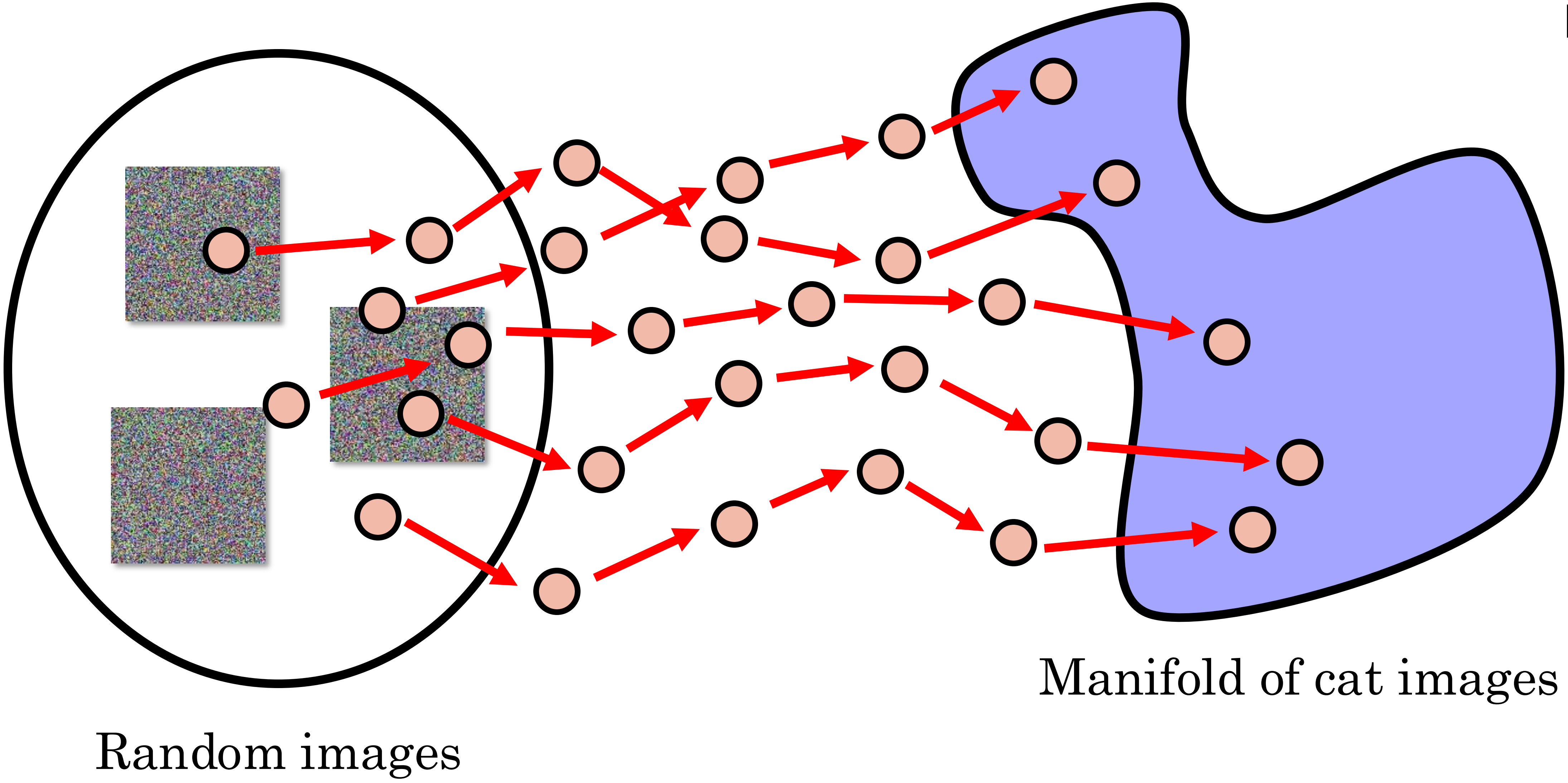
2474272

Slide concept: Steve Seitz



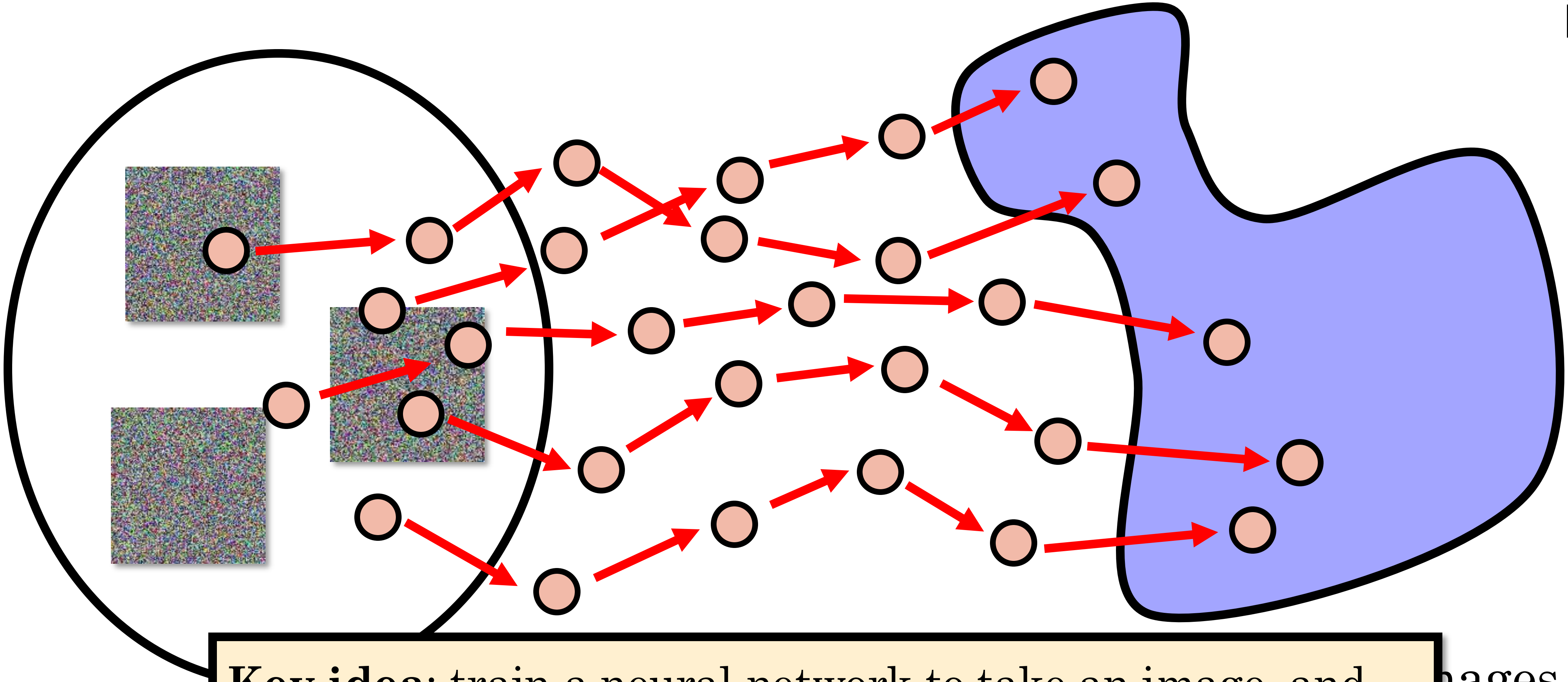
2474272

Slide concept: Steve Seitz



Slide concept: Steve Seitz

2474272

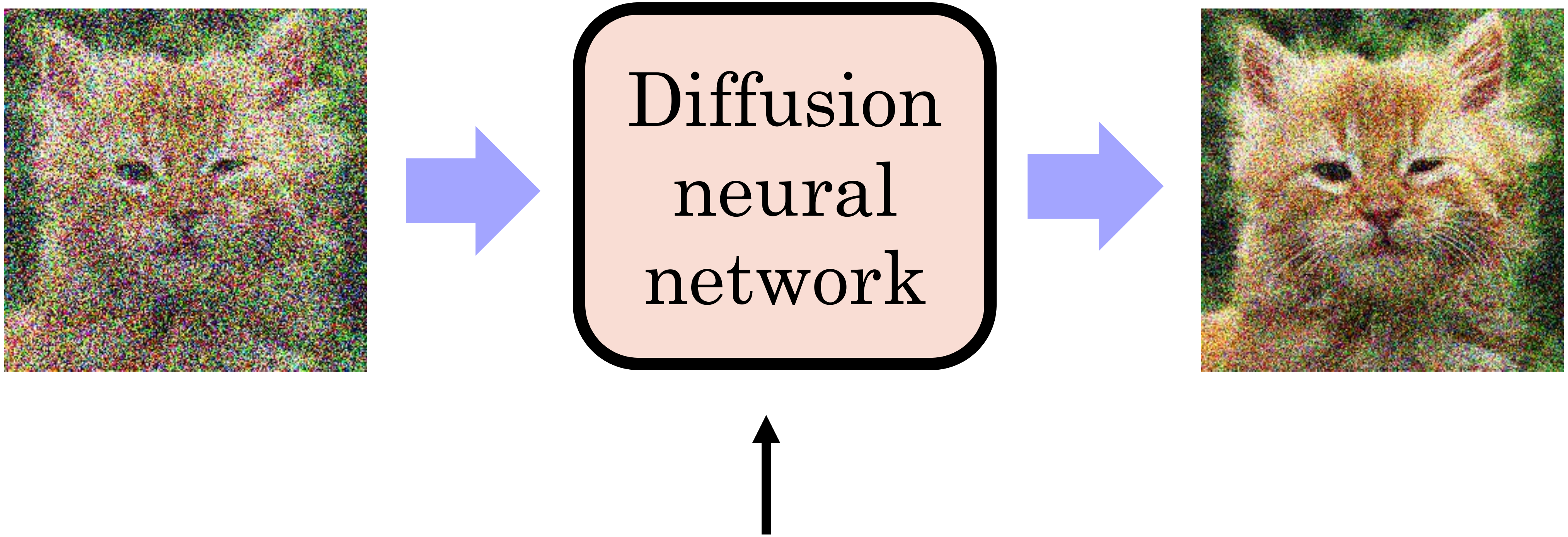


Key idea: train a neural network to take an image, and predict the arrows above; that is, predict to convert a noisy image to a slightly less noisy image that is closer to the desired image manifold, using the example above to train.

Slide concept: Steve Seitz

Denoising Diffusion Neural Network

2474272

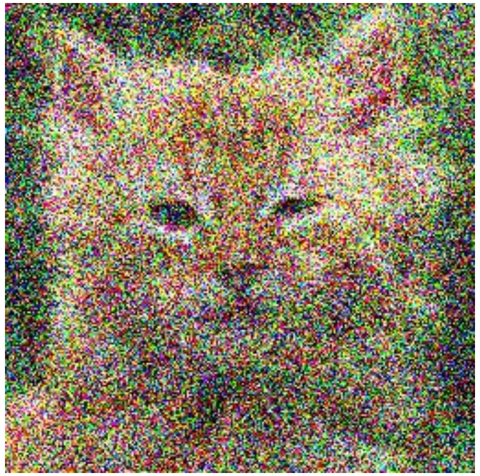


This network can be a U-Net or
other image-to-image network.

Denoising Diffusion Neural Network

2474272

Timestep t (from say 0 to 999) is an additional input to the network (positionally encoded).



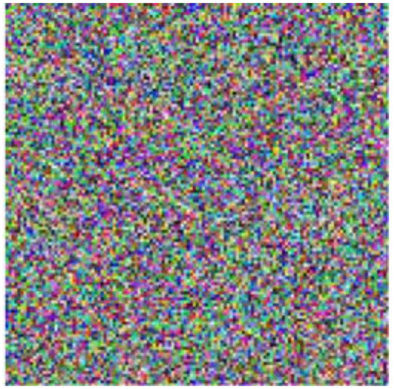
t

Diffusion neural network

A basic diffusion approach will run this denoising for many timesteps (e.g., $T = 1000$ steps)

Sampling Many Outputs

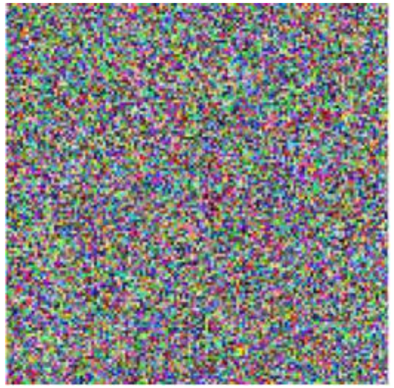
2474272



Noise 1

Output image 1

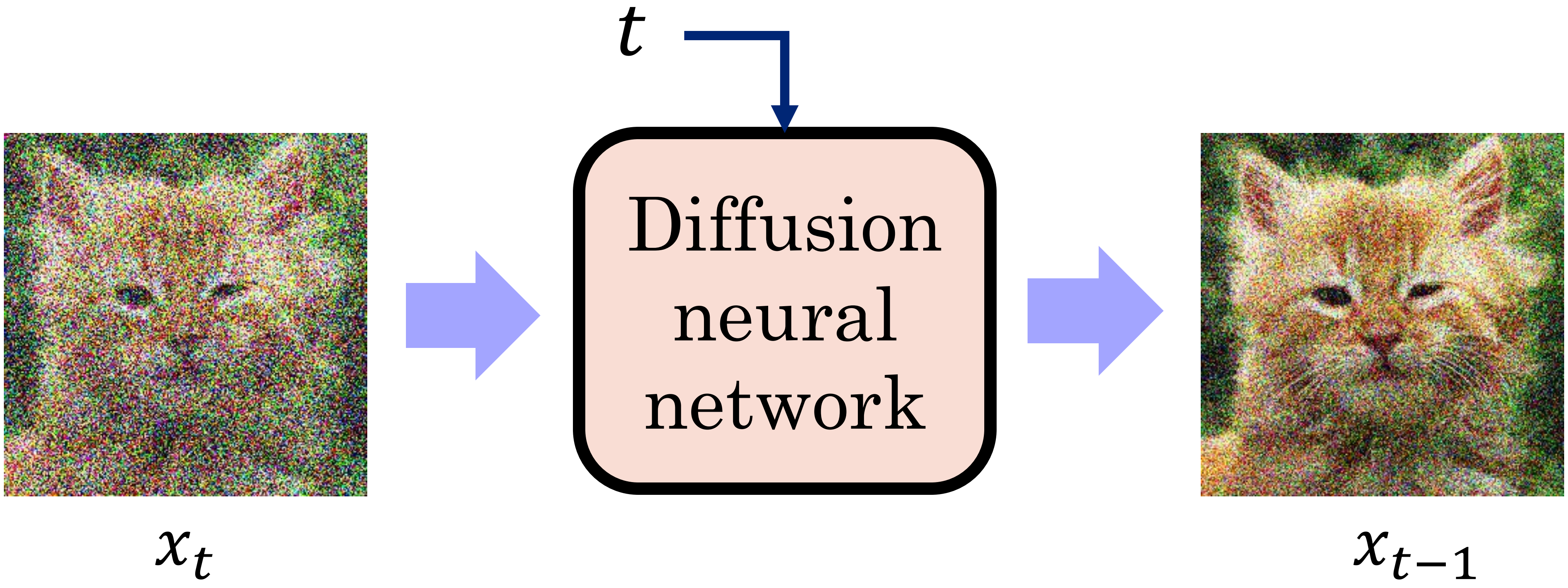
“Walking from random images towards the manifold of natural images”



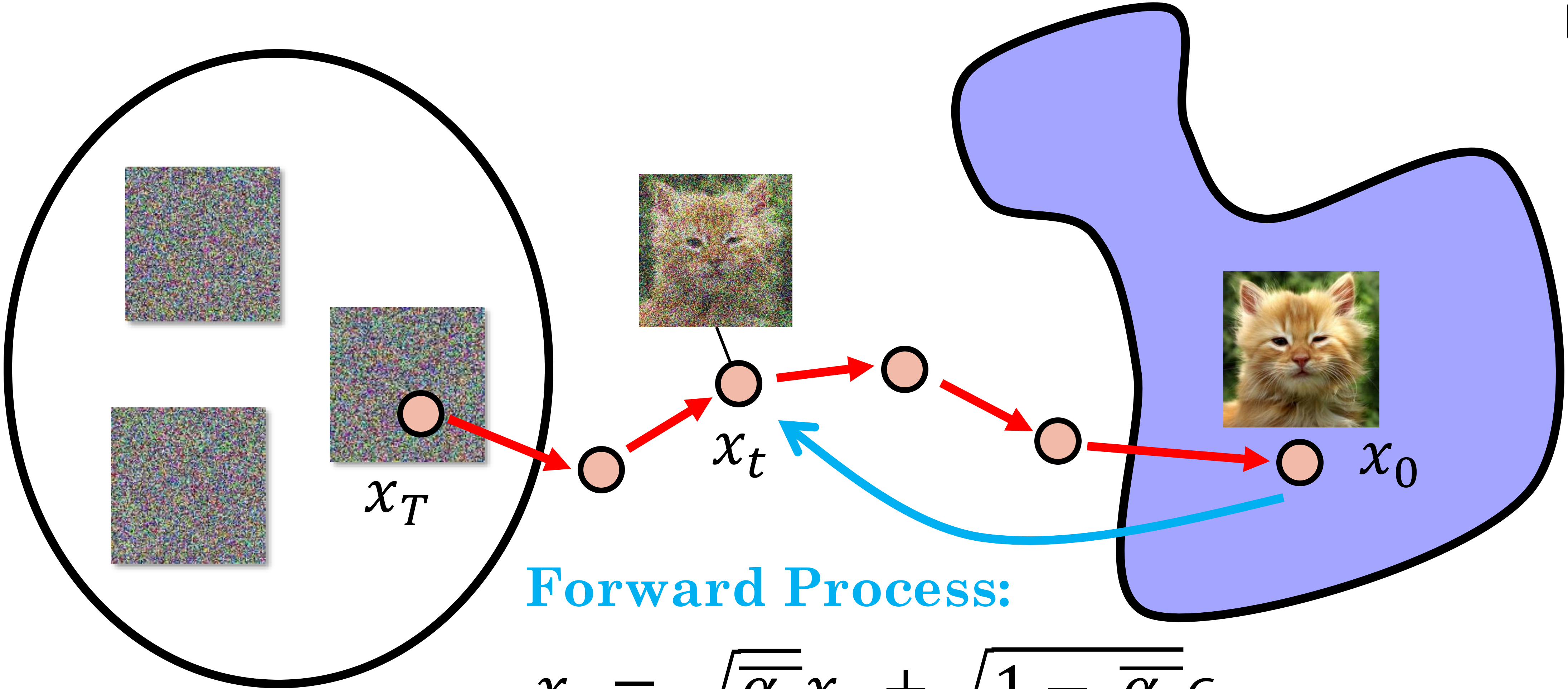
Noise 2

Output image 2

Denoising Diffusion Neural Network



2474272



Forward Process:

$$x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1 - \overline{\alpha}_t}\epsilon$$

Random images

Noise coefficient per time step

$\epsilon \sim \mathcal{N}(0,1) \longrightarrow$ Gaussian noise

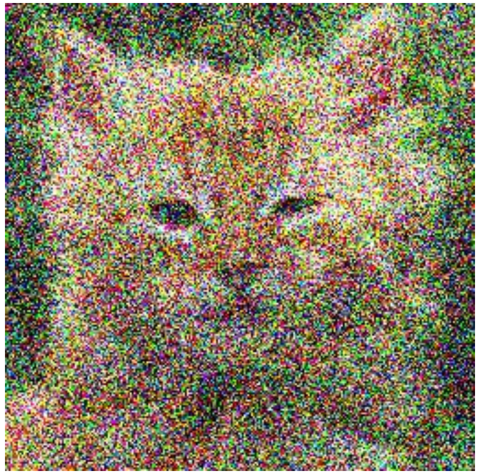
Slide concept: Steve Seitz

In Practice We Predict the Noise

2474272

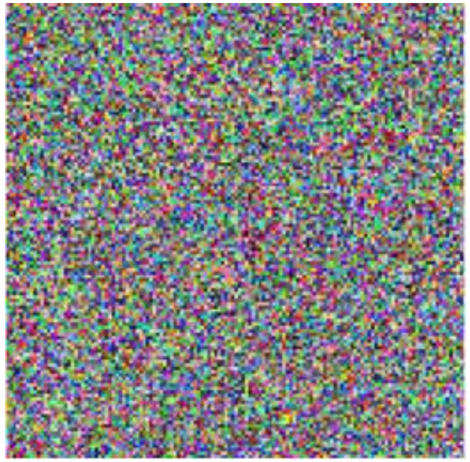
$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\,\epsilon$$

t



x_t

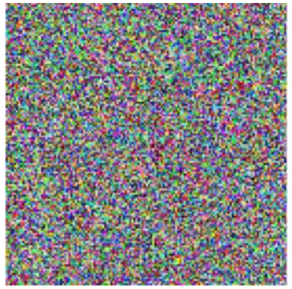
Diffusion neural network



ϵ

$=$

$-$

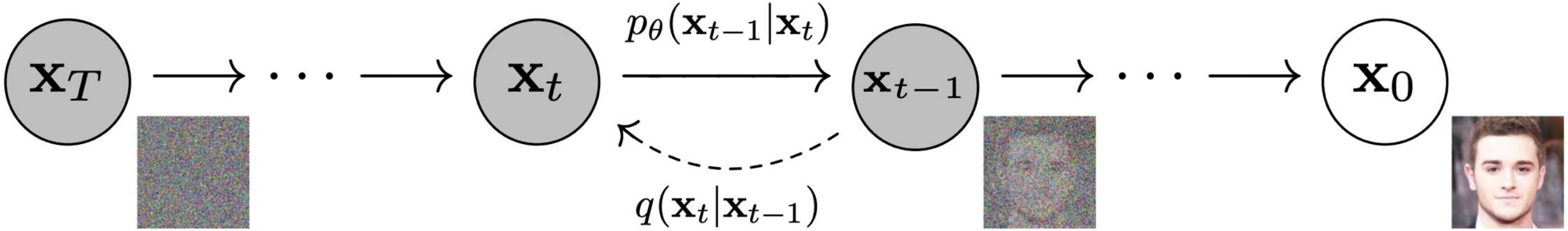


$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t}}\,\epsilon$$

Training a Diffusion Model

2474272

$$x_t = \sqrt{\overline{\alpha_t}}x_0 + \sqrt{1 - \overline{\alpha_t}}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t}\sqrt{1 - \overline{\alpha_t}}}\epsilon$$



Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

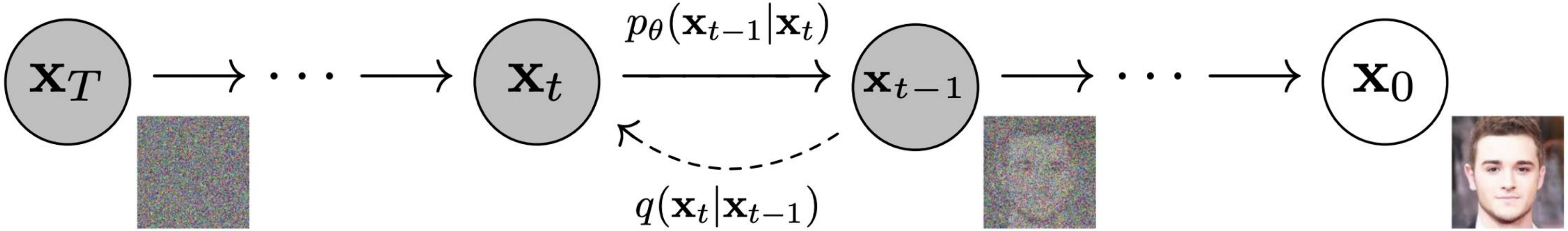
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$

6: **until** converged

Training a Diffusion Model

2474272

$$x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1 - \overline{\alpha}_t}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t}\sqrt{1 - \overline{\alpha}_t}}\epsilon$$



Algorithm 1 Training

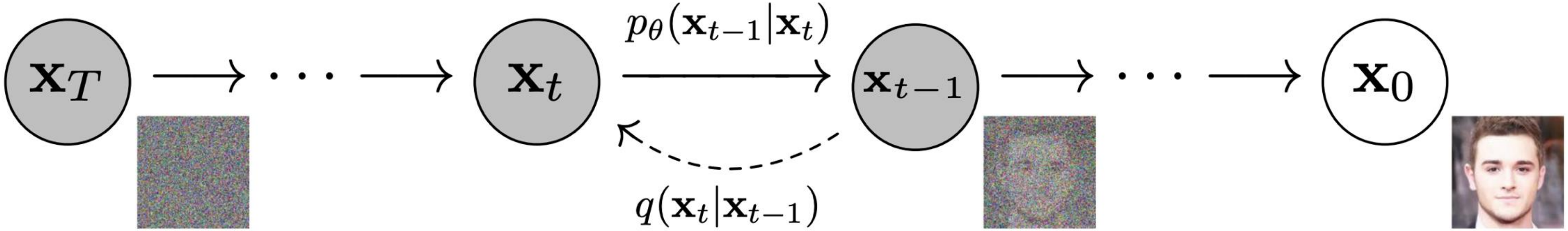
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Sample x_t by adding noise to the clean image x_0

Training a Diffusion Model

2474272

$$x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1-\alpha_t}{\sqrt{\alpha_t}\sqrt{1-\overline{\alpha}_t}}\epsilon$$



Algorithm 1 Training

1: **repeat**
2: $\quad \mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $\quad t \sim \mathrm{Uniform}(\{1, \dots, T\})$
4: $\quad \boxed{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: $\quad$ Take gradient descent step on

$$\nabla_\theta \left\| \boxed{\boldsymbol{\epsilon}} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$

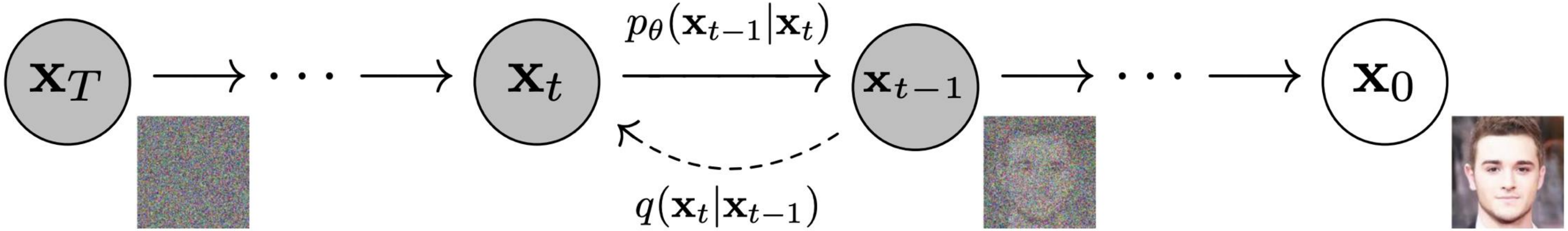
6: **until** converged

Noise

Training a Diffusion Model

2474272

$$x_t = \sqrt{\overline{\alpha_t}}x_0 + \sqrt{1-\overline{\alpha_t}}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1-\alpha_t}{\sqrt{\alpha_t}\sqrt{1-\overline{\alpha_t}}}\epsilon$$



Algorithm 1 Training

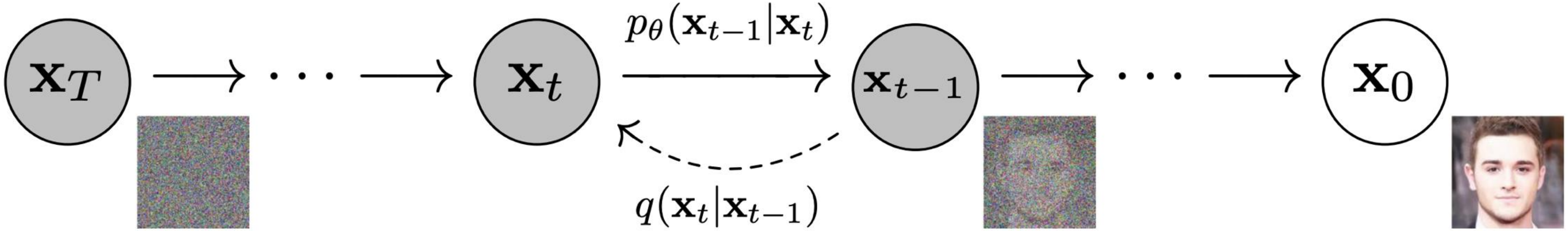
repeat
  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
  $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boxed{\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}}, t) \right\|^2$$
until converged

Noisy image

Training a Diffusion Model

2474272

$$x_t = \sqrt{\overline{\alpha_t}}x_0 + \sqrt{1 - \overline{\alpha_t}}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t}\sqrt{1 - \overline{\alpha_t}}}\epsilon$$



Algorithm 1 Training

repeat
  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
  $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  Take gradient descent step on

$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boxed{\boldsymbol{\epsilon}_\theta}\left(\boxed{\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}}, t\right) \right\|^2$$

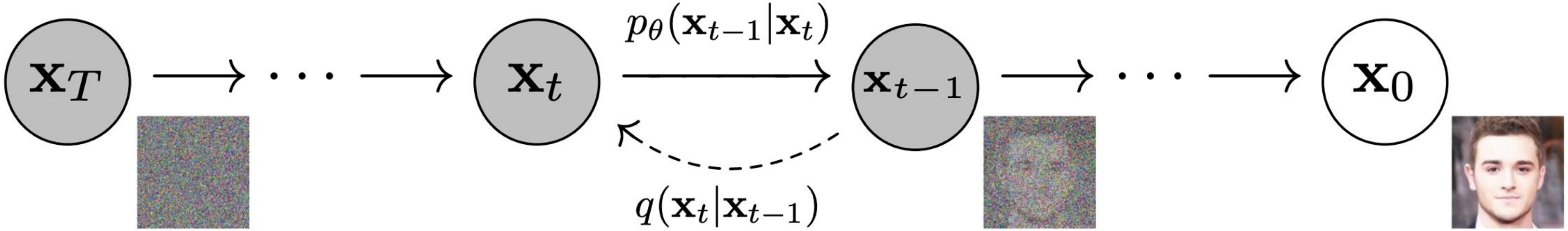
until converged

Model parameters that predict noise ϵ

Noisy image

Training a Diffusion Model

$$x_t = \sqrt{\overline{\alpha_t}}x_0 + \sqrt{1 - \overline{\alpha_t}}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t}\sqrt{1 - \overline{\alpha_t}}}\epsilon$$



Algorithm 1 Training

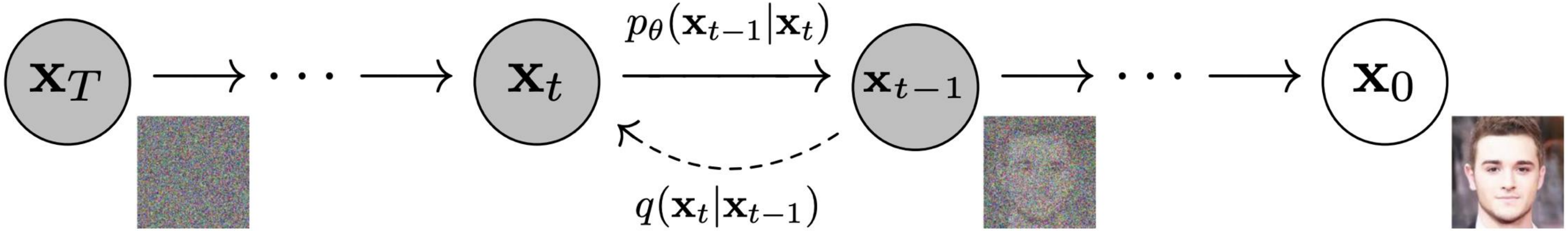
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Make the diffusion model better at de-noising the image.

Sampling from a Diffusion Model

2474272

$$x_t = \sqrt{\overline{\alpha_t}}x_0 + \sqrt{1-\overline{\alpha_t}}\epsilon \qquad x_{t-1} = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1-\alpha_t}{\sqrt{\alpha_t}\sqrt{1-\overline{\alpha_t}}}\epsilon$$



Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \boxed{\frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right)} + \boxed{\sigma_t \mathbf{z}}$
5: **end for**
6: **return** $\mathbf{x}_0$

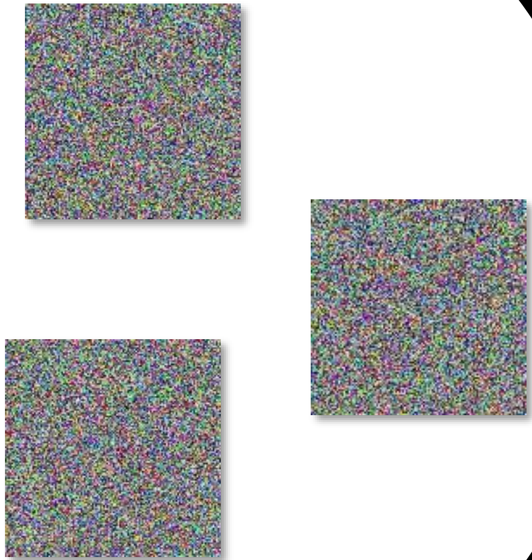
Remove a little bit of noise

Add back a bit of noise to avoid converging to average

Why does the denoising process take many small steps instead of a few big steps?

2474272

cat
images



Random images

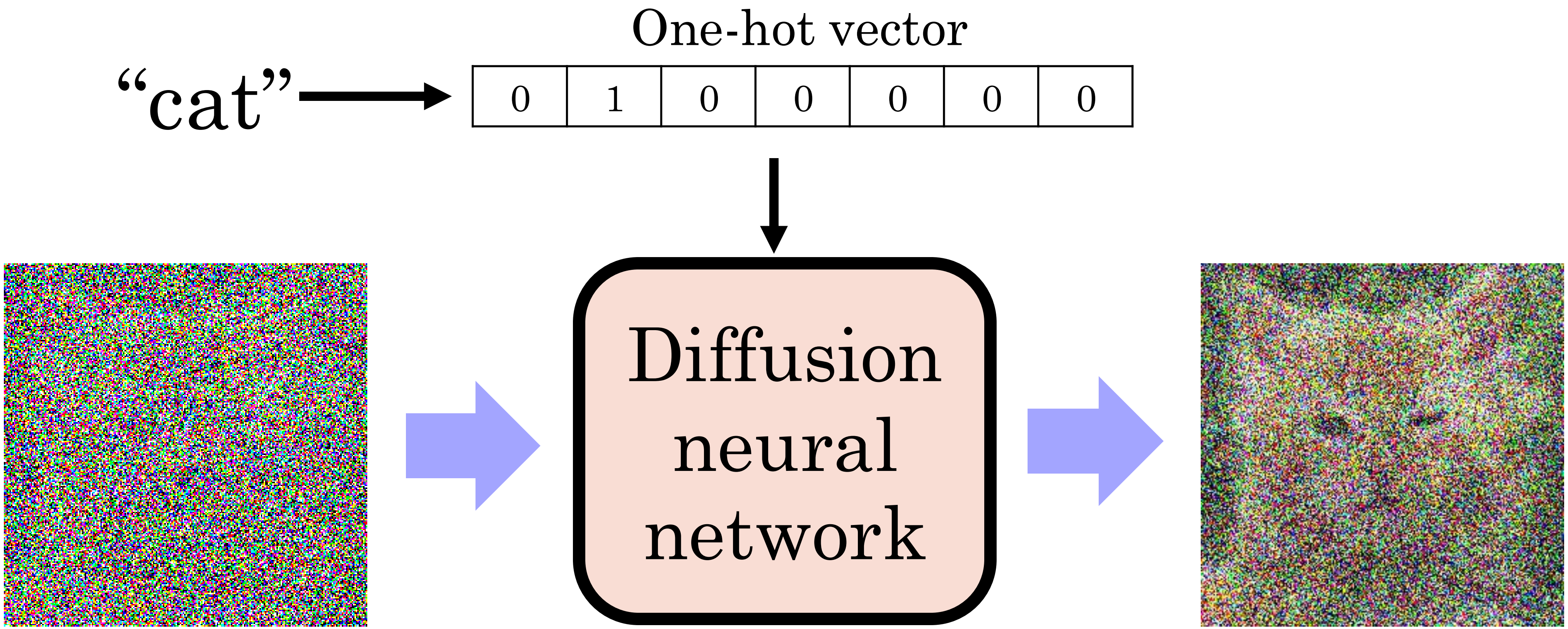
How can we avoid training a separate diffusion network for each concept?

llama
images

Slide concept: Steve Seitz

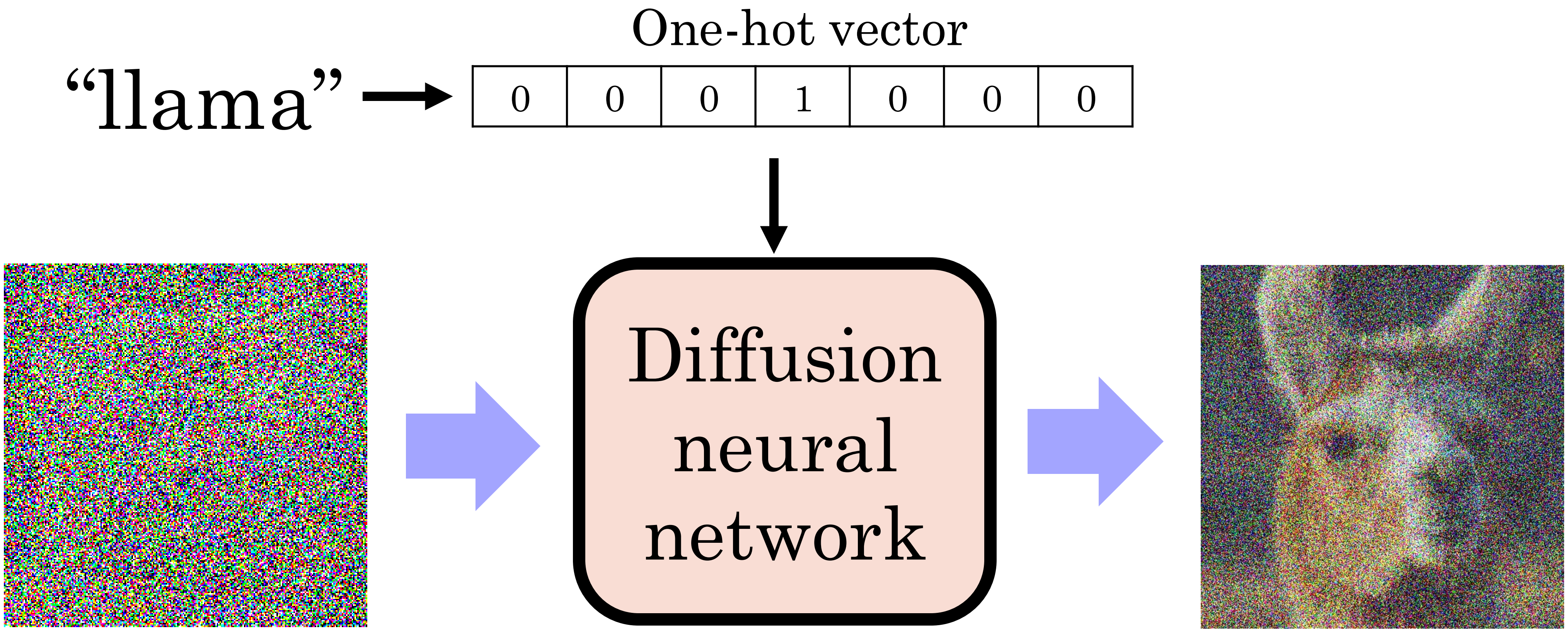
Idea 1: Add a Class Label as Conditioning

2474272



Idea 1: Add a Class Label as Conditioning

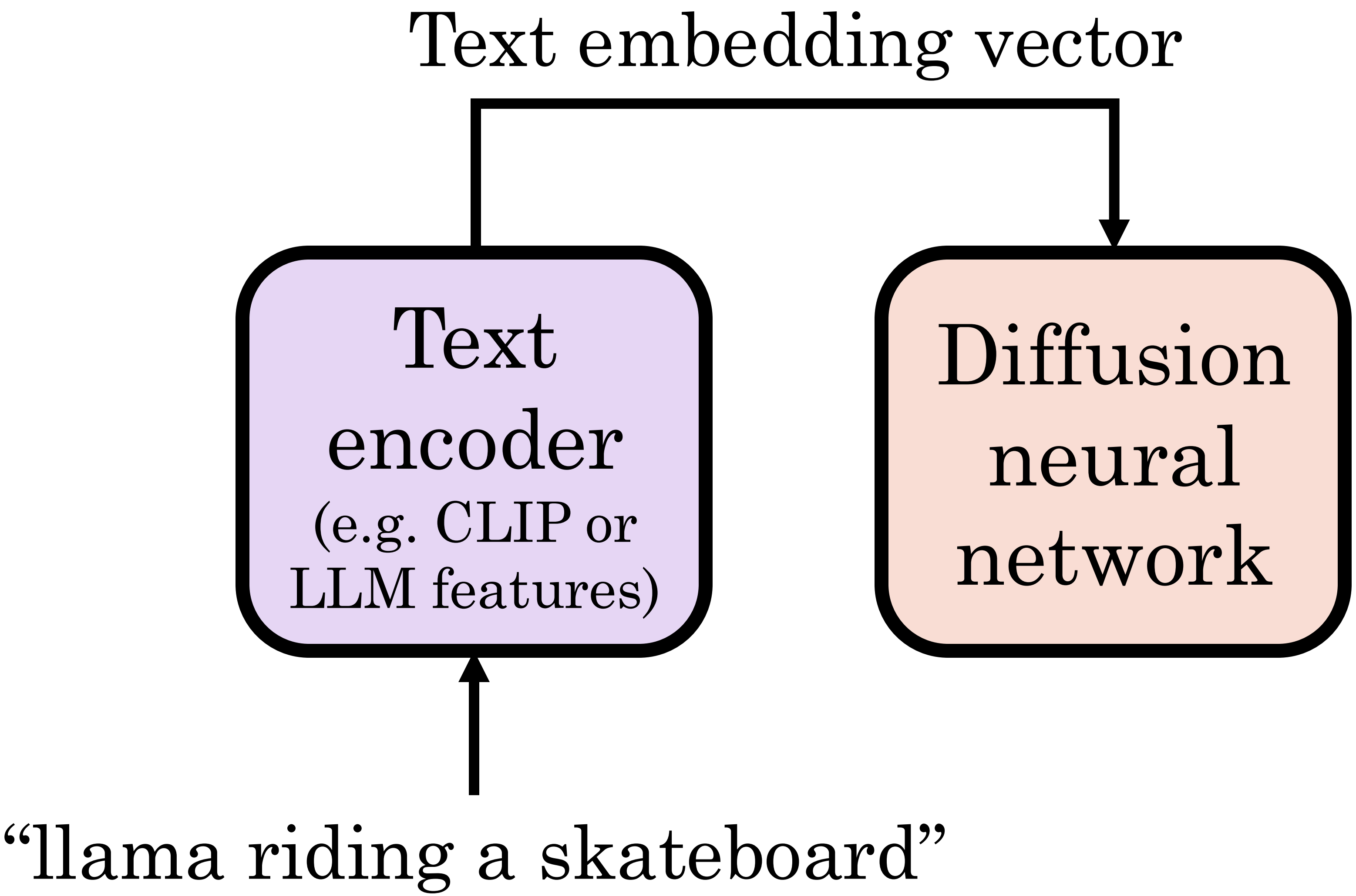
2474272



"Class conditioning" only allows for generating a fixed number of classes.

Idea 2: Add a Text Prompt as a Condition

2474272

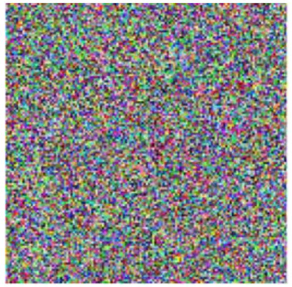


Idea 3: Use an Image Classifier to Guide the Denoising Process ("Classifier Guidance")

x_{t-1}

=

−



+

Update image that makes the input look more like a cat

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1-\alpha_t}{\sqrt{\alpha_t}\sqrt{1-\overline{\alpha_t}}}\epsilon$$

Idea 3: Use an Image Classifier to Guide the Denoising Process ("Classifier Guidance")

2474272

Update image that makes the input look more like a cat

$$= \nabla F_\theta(x_t | y = \text{"}cat\text{"})$$

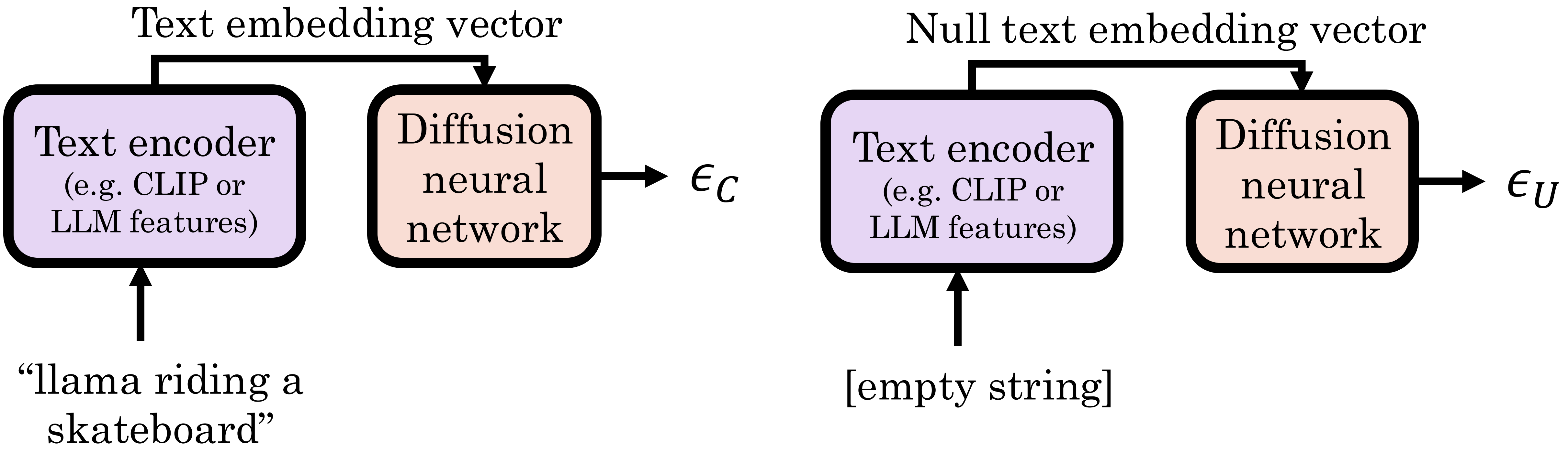
Image classifier such as AlexNet trained on ImageNet but fine-tuned on noisy images

We need to train on noisy images and it requires a separate network and additional memory

$$\tilde{\epsilon}(x_t, t, y) = \epsilon_\theta(x_t, t) - \gamma \nabla_{x_t} \log p(x_t | y)$$

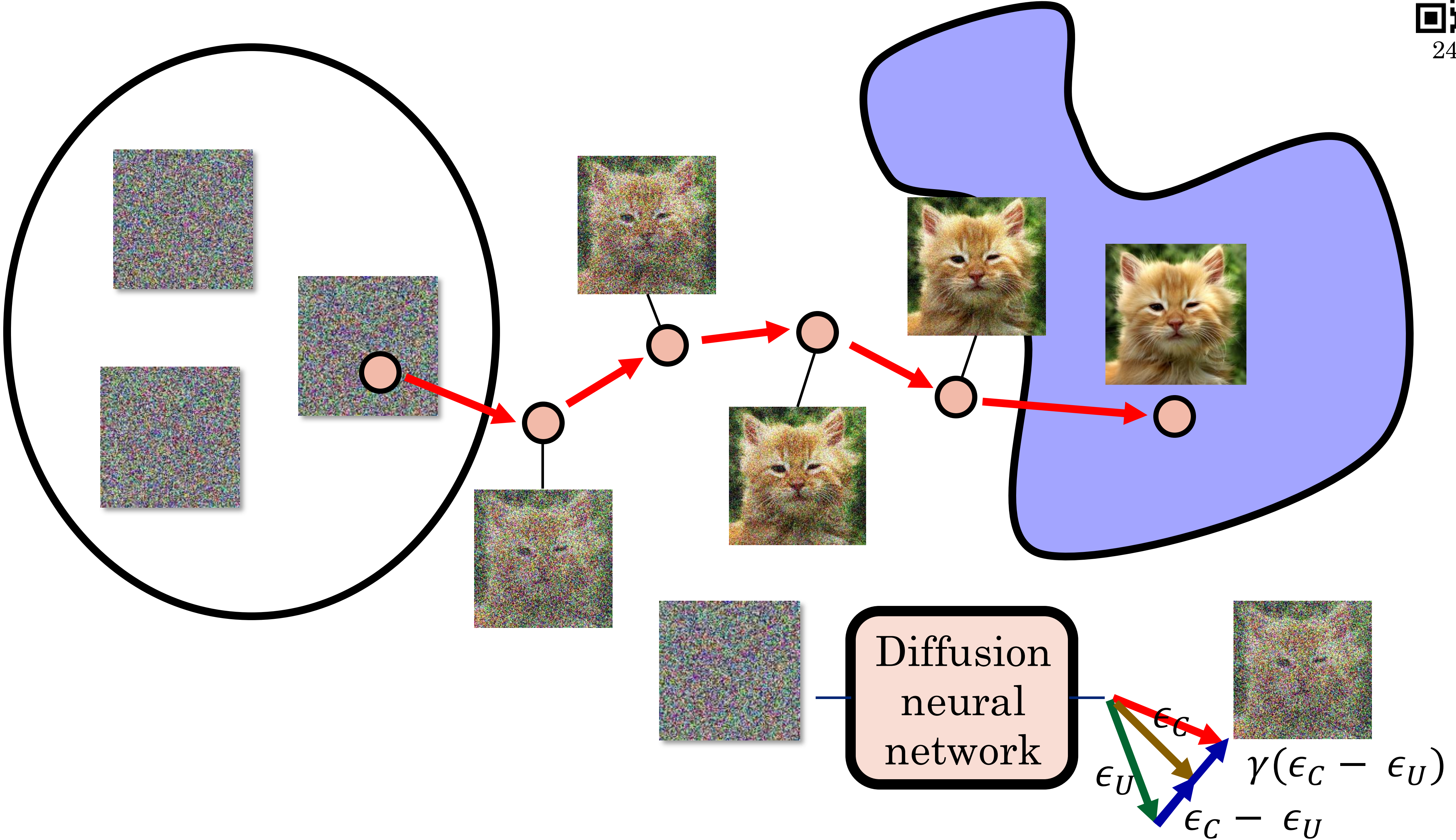
Idea 4: Classifier-Free Guidance (CFG)

2474272



Mixed conditional and unconditional noise:

$$\epsilon = \epsilon_U + \gamma(\epsilon_C - \epsilon_U) \text{ for } \gamma \geq 0$$



Diffusion
neural
network
ϵ_C
ϵ_U
$\gamma(\epsilon_C - \epsilon_U)$
$\epsilon_C - \epsilon_U$

2474272

Classifier-Free Guidance (CFG)

2474272

$\gamma = 0$ → High γ

Non-guided samples

$$\epsilon = \epsilon_U + \gamma(\epsilon_C - \epsilon_U) \text{ for } \gamma > 1$$

Classifier-free diffusion guidance: https://arxiv.org/pdf/2207.12598

Curious Property of Diffusion

2474272

- We are training the model to reconstruct the training set
  - But it fails!
  - Instead, it generate novel images
  - Which is what makes it great
- Perhaps it models images as textures
  - Keeping important correlations and throwing away the rest
  - But we don't know the "model space" of these textures

Lecture 25

Contrastive Learning and Diffusion Models

Reading: Partially covered in Chapter 20 of Bishop Deep Learning Textbook